


**UNIVERSIDADE FEDERAL DE LAVRAS**

# **DOCUMENTO DE DESIGN DE SOFTWARE**: Arquitetura, stack, contrato da API, infraestrutura e entrega contínua do sistema de gestão de uma cooperativa de transportes

2026

**UNIVERSIDADE FEDERAL DE LAVRAS**

Projeto de disciplina: API REST e interface web para cooperativa de transportes

# **DOCUMENTO DE DESIGN DE SOFTWARE**: Arquitetura, stack, contrato da API, infraestrutura e entrega contínua do sistema de gestão de uma cooperativa de transportes

Arthur Valadares Campideli
Discente

Júlia Ribeiro Souza
Discente

Lavras
2026

**RESUMO**

Este documento apresenta o projeto (design) do sistema de gestão de uma cooperativa de transportes, cujos requisitos foram formalizados na Especificação de Requisitos de Software. Ele registra as decisões de arquitetura tomadas em uma quinta rodada de elicitação com perguntas de múltipla escolha: quatro repositórios independentes (contrato da API, backend, infraestrutura e front-end), contrato escrito antes do código em OpenAPI e distribuído como pacotes versionados, backend em Java com Spring Boot, Hibernate e Flyway sobre PostgreSQL, front-end em React com TypeScript e Mantine, autenticação própria por *tokens* em *cookies* protegidos, implantação com Docker Compose e TLS, e integração contínua com GitHub Actions, versionamento semântico e publicação de imagens. O documento detalha a estrutura de cada parte, o modelo físico do banco, a estratégia de ramificação, as verificações automatizadas, os requisitos não funcionais revisados, o plano de implementação e os riscos, e lista as alterações propostas à especificação original.

**Palavras-chave:** Design de software; Arquitetura; Contrato de API; OpenAPI; Spring Boot; React; Integração contínua; Docker.

## LISTA DE ILUSTRAÇÕES

Figura 1 – Visão de contêineres (nível 2 do modelo C4, BROWN, [s. d.]). O retângulo delimita o *host* Docker. ..... 5
Figura 2 – Cadeia de artefatos entre os repositórios. ..... 8
Figura 3 – Fluxo de entrega no backend e no front. ..... 10
Figura 4 – Sequência de login, expiração do *token* de acesso e renovação. A SPA renova uma única vez por falha e, se a renovação falhar, volta ao *login* (CA-10). ..... 24

## LISTA DE QUADROS


Quadro 1 Identificadores usados neste documento. . . . . 1
Quadro 2 Alterações propostas à especificação. . . . . 1
Quadro 3 Direcionadores arquiteturais e suas consequências. . . . . 4
Quadro 4 Registro de decisões de design (ADR resumidos). . . . . 5
Quadro 5 Repositórios, responsabilidades e artefatos publicados. . . . . 8
Quadro 6 Modelo de ramos por repositório. . . . . 9
Quadro 7 Regras de proteção (configuração do GitHub e *workflows*). . . . . 9
Quadro 8 Convenções adicionais do contrato. . . . . 12
Quadro 9 Artefatos gerados e publicados pelo contrato. . . . . 13
Quadro 10 Verificações automáticas no repositório do contrato (bloqueiam o PR). . . . . 13
Quadro 11 Stack do backend. . . . . 15
Quadro 12 Responsabilidade das camadas dentro de um módulo. . . . . 16
Quadro 13 Regras de arquitetura verificadas com ArchUnit (falham o CI). . . . . 16
Quadro 14 Onde cada regra de negócio é aplicada e verificada. . . . . 16
Quadro 15 Tradução de exceções em respostas HTTP. . . . . 18
Quadro 16 Variáveis e *secrets* principais. . . . . 19
Quadro 17 Regras para migrações. . . . . 20
Quadro 18 Índices (o PostgreSQL não cria índices automaticamente em chaves estrangeiras). . . . . 22
Quadro 19 Política de backup e de retenção (D37). . . . . 22
Quadro 20 Credenciais e *cookies*. . . . . 24
Quadro 21 Política de senhas e de contas (a validar, PD04 e PD05). . . . . 25
Quadro 22 Stack do front-end. . . . . 27
Quadro 23 Serviços, redes e verificações de saúde. . . . . 30
Quadro 24 TLS por cenário (D29). . . . . 31
Quadro 25 Ganchos locais por repositório. . . . . 33
Quadro 26 Verificações dos repositórios `gcc188-backend` e `gcc188-front`. . . . . 33
Quadro 27 Verificações dos repositórios `gcc188-api-contract` e `gcc188-infra`. . . . . 34
Quadro 28 Ameaças e controles (categorias do OWASP ASVS, OWASP, 2025). . . . . 35
Quadro 29 Níveis de teste. . . . . 36
Quadro 30 Cenários de aceitação adicionais. . . . . 36
Quadro 31 RNF revisados (Δ08) e novos. . . . . 37
Quadro 32 Marcos de implementação. . . . . 39
Quadro 33 Riscos adicionais. . . . . 40
Quadro 34 Pendências e premissas. . . . . 41

## LISTA DE ABREVIATURAS E SIGLAS

ABNT Associação Brasileira de Normas Técnicas
ACME Automatic Certificate Management Environment
ADR Architecture Decision Record
API Application Programming Interface
CA Critério de Aceitação
CI Continuous Integration
CORS Cross-Origin Resource Sharing
CRUD Create, Read, Update, Delete
CSP Content Security Policy
CSRF Cross-Site Request Forgery
DER Diagrama Entidade-Relacionamento
DTO Data Transfer Object
GHCR GitHub Container Registry
HSTS HTTP Strict Transport Security
HTTP Hypertext Transfer Protocol
JPA Jakarta Persistence API
JSON JavaScript Object Notation
JWT JSON Web Token
LGPD Lei Geral de Proteção de Dados Pessoais
OCI Open Container Initiative
RBAC Role-Based Access Control
REST Representational State Transfer
RF Requisito Funcional
RN Regra de Negócio
RNF Requisito Não Funcional
RPO Recovery Point Objective
RTO Recovery Time Objective
SPA Single-Page Application
SQL Structured Query Language
TLS Transport Layer Security

**SUMÁRIO**


**1 INTRODUÇÃO** ..... 1
1.1 PROPÓSITO E ESCOPO ..... 1
1.2 CONVENÇÕES ..... 1
1.3 ALTERAÇÕES EM RELAÇÃO À ESPECIFICAÇÃO ..... 1

**2 DIRECIONADORES E VISÃO GERAL DA ARQUITETURA** ..... 4
2.1 DIRECIONADORES ARQUITETURAIS ..... 4
2.2 VISÃO DE CONTÊINERES ..... 4
2.3 REGISTRO DE DECISÕES ..... 5

**3 REPOSITÓRIOS, RAMIFICAÇÃO E VERSIONAMENTO** ..... 8
3.1 OS QUATRO REPOSITÓRIOS ..... 8
3.2 ESTRATÉGIA DE RAMIFICAÇÃO POR REPOSITÓRIO ..... 8
3.3 PROTEÇÕES DE RAMO ..... 9
3.4 COMMITS, VERSÃO E *CHANGELOG* AUTOMÁTICOS ..... 9
3.5 COMPATIBILIDADE ENTRE REPOSITÓRIOS ..... 11

**4 CONTRATO DA API (`GCC188-API-CONTRACT`)** ..... 12
4.1 ESTRUTURA DO REPOSITÓRIO ..... 12
4.2 CONVENÇÕES DO CONTRATO ..... 12
4.3 GERAÇÃO E PUBLICAÇÃO DE ARTEFATOS ..... 13
4.4 VERIFICAÇÕES DO CONTRATO ..... 13
4.5 EVOLUÇÃO E COMPATIBILIDADE ..... 14

**5 BACKEND (`GCC188-BACKEND`)** ..... 15
5.1 STACK ..... 15
5.2 MÓDULOS E CAMADAS ..... 15
5.3 REGRAS DE NEGÓCIO, TRANSAÇÕES E CONCORRÊNCIA ..... 16
5.4 PERSISTÊNCIA ..... 17
5.5 TRATAMENTO DE ERROS ..... 18
5.6 TEMPO, MOEDA E ARREDONDAMENTO ..... 18
5.7 OBSERVABILIDADE ..... 19
5.8 CONFIGURAÇÃO ..... 19

**6 BANCO DE DADOS** ..... 20
6.1 SERVIDOR E CONVENÇÕES ..... 20
6.2 MIGRAÇÕES COM FLYWAY ..... 20
6.3 AJUSTES AO MODELO FÍSICO ..... 21
6.4 ADMINISTRADOR INICIAL ..... 22
6.5 BACKUP E RETENÇÃO ..... 22

**7 AUTENTICAÇÃO E AUTORIZAÇÃO** ..... 24
7.1 VISÃO GERAL ..... 24
7.2 ROTAÇÃO DO *REFRESH TOKEN* E REVOGAÇÃO ..... 25

7.3 CSRF E CABEÇALHOS DE SEGURANÇA....25
7.4 SENHAS E CONTAS....25
7.5 AUTORIZAÇÃO....26
7.6 RECUPERAÇÃO DE SENHA POR E-MAIL....26
**8 FRONT-END (`GCC188-FRONT`)**....27
8.1 STACK....27
8.2 ORGANIZAÇÃO DO CÓDIGO....28
8.3 DADOS E CONTRATO....28
8.4 SESSÃO E PERMISSÕES NA INTERFACE....28
8.5 TELAS PADRONIZADAS....29
8.6 PROTOTIPAGEM SEM BACKEND....29
8.7 ENTREGA....29
**9 INFRAESTRUTURA E IMPLANTAÇÃO (`GCC188-INFRA`)**....30
9.1 ESTRUTURA DO REPOSITÓRIO....30
9.2 SERVIÇOS DO COMPOSE....30
9.3 ROTEAMENTO, CABEÇALHOS E TLS....31
9.4 SEGREDOS E CONFIGURAÇÃO....31
9.5 CAMINHO PARA A NUVEM....32
9.6 OBSERVABILIDADE OPCIONAL....32
**10 INTEGRAÇÃO CONTÍNUA**....33
10.1 GANCHOS LOCAIS....33
10.2 PIPELINES POR REPOSITÓRIO....33
10.3 PUBLICAÇÃO DE IMAGENS E PACOTES....34
**11 SEGURANÇA: VISÃO CONSOLIDADA**....35
**12 TESTES E REQUISITOS NÃO FUNCIONAIS REVISADOS**....36
12.1 ESTRATÉGIA DE TESTES....36
12.2 CRITÉRIOS DE ACEITAÇÃO ADICIONAIS....36
12.3 REQUISITOS NÃO FUNCIONAIS REVISADOS E NOVOS....37
**13 PLANO DE IMPLEMENTAÇÃO**....39
**14 RISCOS DE DESIGN**....40
**15 PENDÊNCIAS E PREMISSAS DE DESIGN**....41
**REFERÊNCIAS**....42

# 1 INTRODUÇÃO

## 1.1 PROPÓSITO E ESCOPO

Este documento é o artefato de saída da fase de **Projeto** do modelo em cascata adotado na Especificação de Requisitos de Software (doravante *especificação*). Ele traduz os requisitos aprovados em decisões de arquitetura e de engenharia: organização em quatro repositórios, contrato da API, estrutura do backend e do front-end, persistência e migrações, autenticação, infraestrutura, integração contínua, estratégia de testes e plano de implementação. A descrição segue a lógica da ISO/IEC/IEEE 42010 (ISO/IEC/IEEE, 2022): cada visão responde a preocupações de partes interessadas identificadas, e cada decisão relevante é registrada no formato de *Architecture Decision Record* (NYGARD, 2011).

As decisões resultam da quinta rodada de elicitação, conduzida com o mesmo método das quatro rodadas anteriores (perguntas fechadas de múltipla escolha, 42 perguntas) e consolidada em Seção 2.3. O documento **não repete** os requisitos: ele os referencia por identificador (RF, RN, RNF, CA). Quando uma decisão altera ou refina o texto da especificação, a mudança é registrada em Seção 1.3 e deve ser incorporada à especificação pelo fluxo de gestão de mudanças dela.

Ficam fora deste documento o código-fonte, o projeto visual das telas e a escolha de um provedor de nuvem (apenas o caminho de migração é descrito, Seção 9.5).

## 1.2 CONVENÇÕES

Quadro 1 – Identificadores usados neste documento.

| Prefixo | Significado |
|---|---|
| `D01`… | Decisão de design, registrada em Seção 2.3. Decisões D01 a D37 vêm das respostas do stakeholder; D38 em diante são propostas do projetista, a validar. |
| `Δ01`… | Alteração proposta à especificação (Seção 1.3). |
| `PD01`… | Pendência ou premissa de design a confirmar (Capítulo 15). |
| RF, RN, RNF, CA, UC | Identificadores da especificação, usados sem redefinição. |
| `ORG` | Nome da organização no GitHub, ainda a definir (PD01). |

Fonte: Elaborado pelos autores.

## 1.3 ALTERAÇÕES EM RELAÇÃO À ESPECIFICAÇÃO

A nova stack contradiz ou refina pontos da especificação. O quadro abaixo lista cada ponto; a especificação deve ser revisada em ramo `docs/*` após a aprovação deste documento.

Quadro 2 – Alterações propostas à especificação.

| ID | Item afetado | Alteração | Motivo |
|---|---|---|---|
| Δ01 | P6; seção 2.2 (um repositório com `api/` e `web/`). | Quatro repositórios: `gcc188-api-contract`, `gcc188-backend`, `gcc188-front` e `gcc188-infra`. Commits passam a usar o módulo como escopo (por exemplo `feat(contratos): ...`). | D03. |
| Δ02 | Seção 2.2 (`main`, `release/*`, `hotfix/*`). | Ramo principal chamado `master`. Em backend e front, `master` só recebe PR de `develop` (ou do PR de release automatizado); `release/*` é substituído pelo PR de release; *hotfix* entra por `develop`. Contrato e infraestrutura usam *trunk-based* protegido. | D04, D30. |
| Δ03 | RNF-02 (*token* com expiração máxima de 60 min). | *Token* de acesso de 15 min mais *refresh token* rotativo de 7 dias, persistido. | D21. |
| Δ04 | RNF-19 e seção 9.1 (CORS restrito à origem do front). | Front e API na mesma origem atrás de um proxy reverso; CORS não é habilitado. *Tokens* em *cookies* `HttpOnly` com `SameSite=Strict` e proteção CSRF. | D21, D28. |
| Δ05 | Modelo de dados (seis tabelas). | Tabela técnica `refresh_token` (7ª). Colunas `versao` em `usuario`, `contrato`, `pagamento` e `despesa`; `deve_trocar_senha`, `falhas_login` e `bloqueado_ate` em `usuario`. | D13, D21, D22. |
| Δ06 | Dicionário de dados (`ENUM`, `TIMESTAMP`, `DECIMAL`). | `ENUM` vira `VARCHAR` com `CHECK`; `TIMESTAMP` vira `TIMESTAMPTZ`; `DECIMAL(12,2)` é escrito `NUMERIC(12,2)` (sinônimo no PostgreSQL). | D18, D19. |
| Δ07 | Contrato da API (seção 10). | Novos recursos: `POST /auth/refresh`, `POST /auth/logout`, `GET /auth/me` e `POST /usuarios/{id}/redefinicao-senha`. Campo `versao` nos recursos com concorrência otimista. Corpo de lista padronizado (Seção 4.2). | D13, D21, D22. |
| Δ08 | RNF-10, RNF-11 e RNF-20. | Metas revisadas para valores mensuráveis e realistas (Seção 12.3). | D36. |
| Δ09 | RNF-12 (arquitetura em camadas). | Pacotes por módulo, com camadas dentro de cada módulo; regras verificadas por ArchUnit. | D10. |
| Δ10 | RNF-15 (portabilidade). | Ambiente completo definido por Docker Compose no repositório de infraestrutura. | D27. |
| Δ11 | RN03 (arredondamento a 2 casas) e P5 (atraso derivado). | Arredondamento `HALF_UP`; "hoje" calculado no fuso `America/Sao_Paulo` com `Clock` injetado. | D40. |
| Δ12 | Fora de escopo e P1. | Mantém-se fora de escopo o envio de e-mail. A recuperação de senha é feita pelo administrador, que define | D22. |

| ID | Item afetado | Alteração | Motivo |
|---|---|---|---|
| | | uma senha temporária; a troca é obrigatória no primeiro acesso. | |

Fonte: Elaborado pelos autores.

# 2 DIRECIONADORES E VISÃO GERAL DA ARQUITETURA

## 2.1 DIRECIONADORES ARQUITETURAIS

Quadro 3 – Direcionadores arquiteturais e suas consequências.

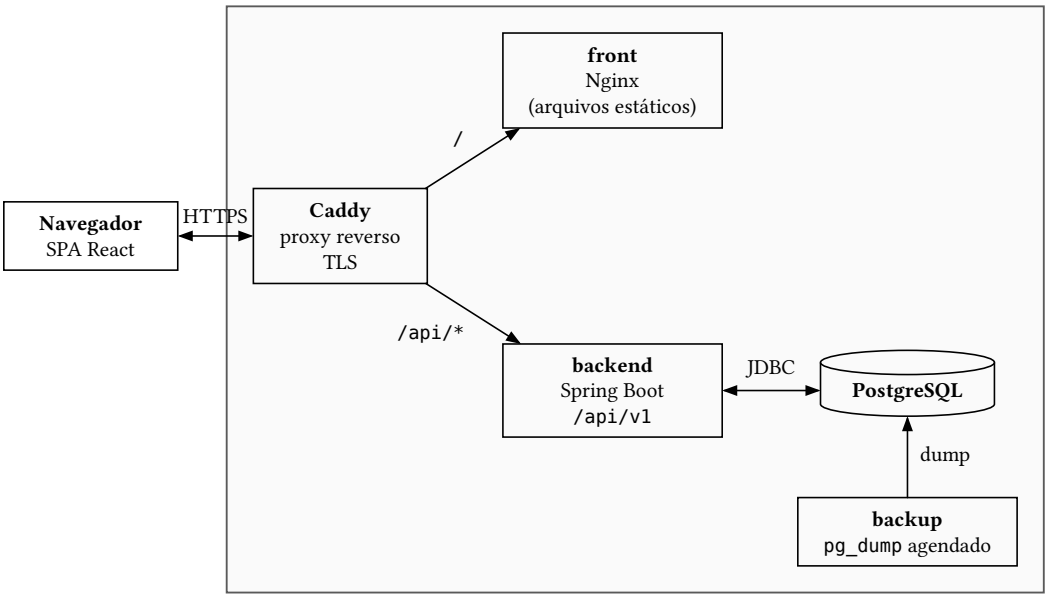
| # | **Direcionador** | **Origem** | **Consequência no design** |
|---|---|---|---|
| 1 | Facilidade de desenvolvimento e de prototipagem da interface. | Objetivo do projeto; R7. | React com Mantine; cliente e *mocks* gerados do contrato; telas padronizadas de listagem, formulário e detalhe. |
| 2 | Reaproveitar tecnologia consolidada nas partes difíceis (interface, autenticação, migrações, TLS). | Objetivo do projeto. | Mantine; Spring Security com biblioteca Nimbus para JWT; Flyway; MapStruct; Orval; Caddy com certificados automáticos. |
| 3 | Integridade financeira. | RN03 a RN07; RNF-04 e RNF-05. | `NUMERIC(12,2)`, `CHECK`, transações atômicas, bloqueio otimista e teste automatizado de cada RN. |
| 4 | Escopo de dados por papel. | RN10; RNF-03. | RBAC no controlador e restrição por propriedade na consulta; recurso fora do escopo responde 404. |
| 5 | Quatro partes independentes. | Decisão do stakeholder. | Repositórios separados; contrato como fonte de verdade; artefatos versionados e publicados. |
| 6 | TLS em todos os cenários. | Decisão do stakeholder. | Proxy reverso com TLS; certificado local (mkcert) e ACME em produção. |
| 7 | Integração contínua e entrega reproduzível. | Decisão do stakeholder. | GitHub Actions; versões e *changelog* automáticos; imagens no GHCR. |
| 8 | Prazo acadêmico e equipe de duas pessoas. | R5. | Preferência por escolhas convencionais e automação que reduza o custo de manter quatro repositórios. |

Fonte: Elaborado pelos autores.

## 2.2 VISÃO DE CONTÊINERES

O sistema em execução é composto por cinco contêineres em um único *host* Docker. Somente o proxy publica portas; o banco e o backend não são alcançáveis de fora (Capítulo 9).

Figura 1 – Visão de contêineres (nível 2 do modelo C4, BROWN, [s. d.]). O retângulo delimita o *host* Docker.




Fonte: Elaborado pelos autores.

## 2.3 REGISTRO DE DECISÕES

O quadro resume as decisões; as justificativas detalhadas estão nos capítulos seguintes. A coluna *Pergunta* remete à numeração da quinta rodada (Q1 a Q42).

Quadro 4 – Registro de decisões de design (ADR resumidos).

| ID | Pergunta | Decisão | Alternativas descartadas |
|---|---|---|---|
| `D01` | Q1 | O design cobre decisões, diagramas, modelo físico, plano de implantação e testes. | Só ADRs. |
| `D02` | Q2 | Documento Typst em padrão ABNT, separado da especificação e referenciando-a. | Novos capítulos na especificação. |
| `D03` | Q3, Q6 | Quatro repositórios no GitHub: `gcc188-api-contract`, `gcc188-backend`, `gcc188-infra` e `gcc188-front`. O custo de coordenação não limita a decisão. | Repositório único. |
| `D04` | Q4 | Git Flow adaptado em backend e front; *trunk-based* protegido em contrato e infraestrutura (Seção 3.2). | Git Flow em todos; *trunk-based* sem proteções. |
| `D05` | Q5 | O contrato é a fonte de compatibilidade; backend e front declaram a versão que implementam (Seção 3.5). | *Release train* pelo repositório de infraestrutura. |
| `D06` | Q7 | Contrato escrito antes do código (*contract-first*) em OpenAPI. | Gerar o contrato a partir do código (springdoc). |
| `D07` | Q8, Q9 | Pacotes publicados: Maven (DTOs e interfaces geradas, consumidos pelo backend) e npm (documento OpenAPI e tipos, consumidos pelo front). | *Git submodule*; *download* por tag no *build*. |
| `D08` | Q10 | Spectral, oasdiff, Prism e Schemathesis; versionamento semântico estrito a partir da 1.0.0. | Sem verificação automática de compatibilidade. |
| `D09` | Q11 | Java 25 (LTS) e Spring Boot 4.x (linha estável vigente na data de fixação, PD12). | Java 21; Spring Boot 3.5 (fim do suporte aberto em 30 jun. 2026). |
| `D10` | Q12 | Pacotes por módulo, com camadas dentro de cada módulo. | Camadas globais. |

| ID | Pergunta | Decisão | Alternativas descartadas |
|---|---|---|---|
| D11 | Q13 | Spring Data Specifications para filtros; JPQL ou SQL nativo para relatórios. | Querydsl. |
| D12 | Q14 | MapStruct para entidade e DTO. | Mapeamento manual. |
| D13 | Q15 | Bloqueio otimista (`@Version`). | Bloqueio pessimista. |
| D14 | Q16 | Spring Data Auditing para `criado_em`, `atualizado_em` e autor. | Hibernate Envers. |
| D15 | Q17 | Actuator, logs em JSON e identificador de correlação. | Prometheus e traços distribuídos na v1. |
| D16 | Q18 | PostgreSQL em contêiner próprio. | MySQL ou MariaDB; H2. |
| D17 | Q19 | Flyway. Administrador inicial criado no primeiro *boot* a partir de variáveis de ambiente. | Liquibase; Atlas. |
| D18 | Q20 | `VARCHAR` com `CHECK` no lugar de `ENUM`. | `ENUM` nativo. |
| D19 | Q21 | `BIGINT` com *identity* e `TIMESTAMPTZ`. | UUID. |
| D20 | Q22 | Testes de integração com Testcontainers. | H2 em memória. |
| D21 | Q23, Q24, Q25 | Autenticação própria com Spring Security: JWT de acesso (15 min) e *refresh token* rotativo, ambos em *cookies* `HttpOnly`. | IdP externo; sessão no servidor; *token* em memória. |
| D22 | Q26 | Recuperação de senha por e-mail fora do escopo; o administrador redefine a senha (Seção 7.6). | SMTP na v1. |
| D23 | Q27 | React, TypeScript e Vite. | PHP, Angular, Vue, react-admin. |
| D24 | Q28 | Mantine; React Hook Form, Zod e TanStack Query. | MUI; shadcn/ui. |
| D25 | Q29 | MSW (front) e Prism (contrato) para prototipar sem backend. | Dados fixos no código. |
| D26 | Q30, Q39 | Vitest e Testing Library no front; Playwright no sistema integrado; verificação de acessibilidade e Lighthouse CI. | Somente testes manuais. |
| D27 | Q31 | Somente Docker Compose agora, com configuração pronta para nuvem (Seção 9.5). | Implantação em nuvem na v1. |
| D28 | Q32 | Mesma origem: `/` para o front e `/api` para o backend. | Subdomínios com CORS. |
| D29 | Q33 | TLS com mkcert (desenvolvimento) e ACME no Caddy (produção); terminação no proxy; `sslmode` do banco configurável. | TLS ponta a ponta obrigatório. |
| D30 | Q34 | `bootBuildImage` e GitHub Actions publicam no GHCR; versão, *tag* e *changelog* automáticos; `master` protegida (só `develop` para `master`). | Dockerfile no backend; versão manual. |
| D31 | Q35 | GitHub Secrets no CI e *secrets* do Compose na execução; sem IaC além do Compose. | Terraform ou Ansible. |
| D32 | Q36 | GitHub Actions. | GitLab CI. |
| D33 | Q37 | lefthook e commitlint. | pre-commit (Python); Husky. |
| D34 | Q38, Q39 | Todas as verificações propostas entram como bloqueio (Capítulo 10). | Verificações opcionais. |
| D35 | Q40 | Somente integração contínua; publicação de imagens apenas em *release*. | Implantação contínua. |
| D36 | Q41 | RNF-10, RNF-11 e RNF-20 revisados (Seção 12.3). | Manter as metas originais. |
| D37 | Q42 | Backups com política de retenção e testes de restauração; retenção de logs (Seção 6.5). | Sem backup. |
| D38 | — | JSON com campos em `snake_case`, iguais aos nomes de coluna (CA-02 e CA-09 já usam `taxa_valor`). | `camelCase`. |
| D39 | — | Valores monetários no JSON como número decimal com duas casas; o front não faz aritmética monetária. | Texto decimal. |

| ID | Pergunta | Decisão | Alternativas descartadas |
|---|---|---|---|
| D40 | — | Arredondamento HALF_UP com escala 2 (RN03, RN05); "hoje" calculado no fuso America/Sao_Paulo com Clock injetado. | HALF_EVEN. |
| D41 | — | Eventos de domínio síncronos entre módulos (aprovação e cancelamento) e portas para consultas (encerramento). | Chamadas diretas entre serviços. |
| D42 | — | Caddy como proxy reverso. | Traefik; Nginx com certbot. |

Fonte: Elaborado pelos autores.

# 3 REPOSITÓRIOS, RAMIFICAÇÃO E VERSIONAMENTO

## 3.1 OS QUATRO REPOSITÓRIOS

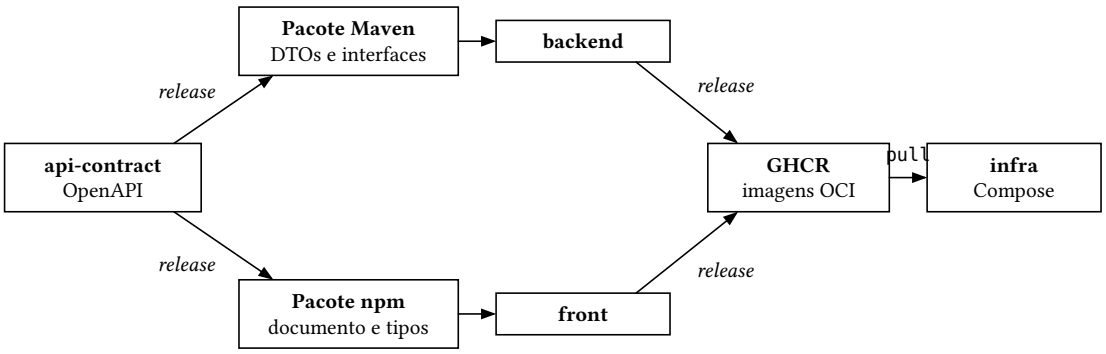
Cada parte do sistema tem ciclo de vida, ferramentas e verificações próprios; por isso vive em repositório separado (D03). A dependência entre eles é sempre mediada por **artefatos versionados** (pacotes e imagens), nunca por código copiado.

Quadro 5 – Repositórios, responsabilidades e artefatos publicados.

| Repositório | Responsabilidade | Conteúdo principal | Artefato publicado |
|---|---|---|---|
| `gcc188-api-contract` | Fonte de verdade da API: recursos, esquemas, erros e segurança. | Documento OpenAPI 3.1, regras Spectral, projeto de geração Java e empacotamento TypeScript. | Pacote Maven (DTOs e interfaces) e pacote npm (documento e tipos), ambos no GitHub Packages. |
| `gcc188-backend` | Implementa o contrato: regras de negócio, persistência, segurança. | Aplicação Spring Boot, migrações Flyway, testes. | Imagem OCI `ghcr.io/ORG/gcc188-backend`. |
| `gcc188-front` | Interface web (SPA) por papel. | Aplicação React, mocks MSW, testes. | Imagem OCI `ghcr.io/ORG/gcc188-front` (Nginx com arquivos estáticos). |
| `gcc188-infra` | Ambiente executável do sistema completo. | Compose, Caddyfile, *scripts* de backup, testes ponta a ponta, perfil opcional de observabilidade. | Nenhum artefato de *runtime*; *tags* marcam conjuntos de versões testados. |

Fonte: Elaborado pelos autores.

Figura 2 – Cadeia de artefatos entre os repositórios.




Fonte: Elaborado pelos autores.

## 3.2 ESTRATÉGIA DE RAMIFICAÇÃO POR REPOSITÓRIO

O stakeholder perguntou se o desenvolvimento em tronco único (*trunk-based*) não seria simplório e sujeito a problemas. A preocupação procede para a forma ingênua (*commits* diretos, sem verificação obrigatória, ramos longos). Já a forma disciplinada, com ramos de vida curta, integração contínua obrigatória e revisão, é associada a equipes de alto desempenho

(FORSGREN; HUMBLE; KIM, 2018). A decisão (D04) é usar essa forma disciplinada onde o repositório não precisa de uma área de estabilização, e manter o Git Flow onde há aplicação executável cujo conjunto de funcionalidades se estabiliza antes da entrega.

Quadro 6 – Modelo de ramos por repositório.

| **Repositório** | **Modelo** | **Regras** |
|---|---|---|
| `gcc188-backend`, `gcc188-front` | Git Flow adaptado | Ramos de longa vida: `master` e `develop`. Ramos curtos: `feature/*`, `fix/*`, `docs/*` e `hotfix/*`, todos criados a partir de `develop` e integrados nela por PR (*squash*, título no padrão *Conventional Commits*). A promoção é um PR de `develop` para `master` (*merge commit*). *Hotfix* entra por `develop` em PR de urgência e é promovido em seguida. |
| `gcc188-api-contract`, `gcc188-infra` | *Trunk-based* protegido | Um único ramo de longa vida, `master`. Ramos curtos (`feat/*`, `fix/*`) com vida de até dois dias úteis, integrados por PR com revisão e verificações obrigatórias. No contrato, a verificação de compatibilidade (oasdiff) funciona como rede de segurança adicional, pois um erro se propaga ao backend e ao front. |

Fonte: Elaborado pelos autores.

Por que não usar Git Flow nos quatro: contrato e infraestrutura não têm uma "versão em estabilização" separada do desenvolvimento, e o ramo `develop` seria apenas um espelho de `master`. Por que não *trunk-based* nos dois principais: o stakeholder pediu `master` somente com PR vindo de `develop` (D30), o que pressupõe o ramo de integração.

## 3.3 PROTEÇÕES DE RAMO

Quadro 7 – Regras de proteção (configuração do GitHub e *workflows*).

| **Ramo** | **Regras** |
|---|---|
| `master` (backend, front) | PR obrigatório; origem somente `develop` ou o ramo do PR de release (verificação `source-branch-guard`); verificações de status obrigatórias; um revisor que não seja o autor; conversas resolvidas; ramo atualizado; sem *push* direto, sem *force-push*, sem exclusão; vale também para administradores. |
| `develop` (backend, front) | PR obrigatório; verificações obrigatórias; um revisor; sem *force-push*; *squash merge* para ramos de trabalho. |
| `master` (contrato, infra) | PR obrigatório; verificações obrigatórias; um revisor; histórico linear (*squash merge*); sem *push* direto. |
| *Tags* `v*` | Criadas apenas pelo *workflow* de release; *tags* protegidas contra alteração. |

Fonte: Elaborado pelos autores.

## 3.4 COMMITS, VERSÃO E *CHANGELOG* AUTOMÁTICOS

Os *commits* seguem o *Conventional Commits* (CONVENTIONAL COMMITS, [s. d.]) com o módulo como escopo e o requisito entre parênteses, por exemplo `feat(contratos):`

`aprovar contrato e gerar pagamentos (RF-CTR-04)`. Um *workflow* valida o título de cada PR e as mensagens (commitlint).

A versão, o *changelog*, a *tag* e a *release* são gerados pelo Release Please (GOOGLEAPIS, [s. d.]), que lê os *commits* integrados em `master`:

- cada *push* em `master` abre ou atualiza um **PR de release** contendo o aumento de versão e o `CHANGELOG.md`;
- o PR é aprovado como qualquer outro; ao ser integrado, o *workflow* cria a *tag* `vX.Y.Z` e a *release* no GitHub, conforme o versionamento semântico (PRESTON-WERNER, 2013);
- a publicação do artefato (imagem ou pacote) é disparada pela *release* publicada;
- um *workflow* abre automaticamente o PR de retorno `master` → `develop` para levar o aumento de versão e o *changelog* de volta (somente backend e front);
- antes da 1.0.0, `feat` incrementa a versão *minor* e mudança incompatível não exige *major* (`bump-minor-pre-major`).

Código 1 – Configuração mínima do Release Please no backend (`release-please-config.json`) e marcação da versão em `gradle.properties`.

```
{
  "$schema": "https://raw.githubusercontent.com/googleapis/release-please/main/schemas/config.
json",
  "release-type": "simple",
  "bump-minor-pre-major": true,
  "packages": {
    ".": {
      "package-name": "gcc188-backend",
      "changelog-path": "CHANGELOG.md",
      "extra-files": ["gradle.properties"]
    }
  }
}
```

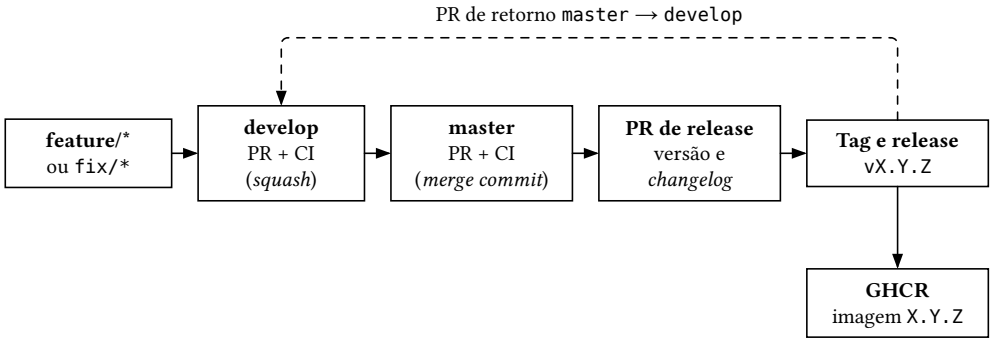
Fonte: Elaborado pelos autores.

Código 2 – Trecho de `gradle.properties` com a marcação lida pelo Release Please.

```
# x-release-please-start-version
version=0.1.0
# x-release-please-end
```

Fonte: Elaborado pelos autores.

Figura 3 – Fluxo de entrega no backend e no front.




Fonte: Elaborado pelos autores.

## 3.5 COMPATIBILIDADE ENTRE REPOSITÓRIOS

O contrato é a fonte de compatibilidade (D05):

- o backend declara a versão do pacote Maven do contrato em `gradle/libs.versions.toml`; o front declara a do pacote npm em `package.json`;
- dentro da mesma versão *major* do contrato, o front funciona com qualquer versão *minor* igual ou superior à que declara, porque mudanças compatíveis só adicionam recursos e campos;
- a promoção de uma nova versão *major* exige PRs coordenados no backend e no front, com período de convivência (Seção 4.5);
- o repositório de infraestrutura fixa, em `versions.env`, o conjunto de versões de imagens que passou nos testes ponta a ponta; uma *tag* de infraestrutura equivale a "um sistema que funciona".

# 4 CONTRATO DA API (`GCC188-API-CONTRACT`)

## 4.1 ESTRUTURA DO REPOSITÓRIO

Código 3 – Estrutura de diretórios do repositório do contrato.

```
gcc188-api-contract/
├── openapi/
│   ├── openapi.yaml           # ponto de entrada: info, servers, security, tags
│   ├── paths/                 # um arquivo por recurso (usuarios.yaml, contratos.yaml, ...)
│   └── components/
│       ├── schemas/           # DTOs e enumerações
│       ├── parameters/        # page, size, sort e filtros comuns
│       ├── responses/         # 400, 401, 403, 404, 409 e 422 (application/problem+json)
│       └── examples/          # exemplos usados pelo Prism e pelo MSW
├── java/                      # projeto Gradle: gera e publica DTOs e interfaces
├── ts/                        # empacota o documento e gera tipos TypeScript
├── .spectral.yaml             # regras de estilo do contrato
├── release-please-config.json
└── .github/workflows/
```

Fonte: Elaborado pelos autores.

## 4.2 CONVENÇÕES DO CONTRATO

O contrato mantém as convenções da seção 10.1 da especificação (base `/api/v1`, comandos de transição por `POST` em sub-recurso, códigos 200, 201, 204, 400, 401, 403, 404, 409 e 422) e acrescenta as seguintes:

Quadro 8 – Convenções adicionais do contrato.

| Tema | Convenção |
|---|---|
| Versão do formato | OpenAPI 3.1. Um arquivo por recurso em `paths/`; esquemas reutilizáveis em `components/`. |
| Nomes (D38) | Campos JSON em `snake_case`, iguais às colunas (`taxa_valor`, `cooperando_id`). `operationId` em `camelCase` com verbo e recurso (`aprovarContrato`). Uma *tag* por módulo. |
| Valores monetários (D39) | Número decimal com duas casas (`multipleOf: 0.01`). O front não soma nem multiplica valores; exibe os valores calculados pela API. |
| Datas | `date` (AAAA-MM-DD) para vencimentos e vigência; `date-time` em UTC para instantes (RNF-14). |
| Enumerações | Textos em maiúsculas, iguais aos valores do banco (`PENDENTE_APROVACAO`). |
| Lista paginada | Corpo `{ content, page, size, total_elements, total_pages }`. Parâmetros `page` (inicia em 0), `size` (padrão 20, máximo 100) e `sort` (somente campos declarados no contrato). |
| Erros | `application/problem+json` conforme a RFC 9457 (NOTTINGHAM; WILDE; DALAL, 2023), com extensões `campos_invalidos` (lista de `campo` e `mensagem`), `codigo_regra` (por exemplo `RN06`) e `correlation_id`. |
| Segurança | Esquema `cookieAuth` (*cookie* `ACCESS_TOKEN`). Requisições de escrita exigem o cabeçalho `X-XSRF-TOKEN` (Capítulo 7). `POST /auth/login` e `POST /auth/refresh` são públicos. |

| Tema | Convenção |
|---|---|
| Concorrência (D13) | `versao` presente nas leituras de usuário, contrato, pagamento e despesa; obrigatório em `PUT`. Divergência responde 409. |
| Exemplos | Todo recurso e todo erro têm ao menos um exemplo; os exemplos alimentam o Prism e o MSW. |

Fonte: Elaborado pelos autores.

Os recursos novos em relação à seção 10.2 da especificação são: `POST /auth/refresh` (renova o par de *cookies*), `POST /auth/logout` (revoga o *refresh token*), `GET /auth/me` (usuário, papel e vínculo, necessário porque o *token* não é legível pelo navegador) e `POST /usuarios/{id}/redefinicao-senha` (administrador define senha temporária).

## 4.3 GERAÇÃO E PUBLICAÇÃO DE ARTEFATOS

A geração acontece no repositório do contrato e os resultados são publicados como pacotes (D07). O backend e o front não geram DTOs a partir de arquivos copiados.

Quadro 9 – Artefatos gerados e publicados pelo contrato.

| Pacote | Conteúdo e geração | Consumo |
|---|---|---|
| Maven `br.ufla.gcc188:cooperativa-api-contract` | Gerador `spring` do OpenAPI Generator em modo somente interfaces (`interfaceOnly`), com `useSpringBoot4` e `useJackson3` (opções disponíveis desde a versão 7.16.0 do gerador). Produz DTOs e uma interface por *tag* (`ContratosApi`, ...). | O backend implementa as interfaces nos controladores; o compilador acusa qualquer divergência com o contrato. |
| npm `@ORG/api-contract` | Documento OpenAPI empacotado em um só arquivo (*bundle*) e tipos TypeScript. | O front gera, em seu próprio repositório, *hooks* do TanStack Query, esquemas Zod e *handlers* MSW com Orval (Seção 8.3). |

Fonte: Elaborado pelos autores.

Essa divisão evita acoplar o contrato à biblioteca de dados do front: o contrato publica o documento; cada consumidor gera o que precisa. A leitura de pacotes do GitHub Packages exige um *token* mesmo em repositórios públicos; no CI, o `GITHUB_TOKEN` basta, e no computador de desenvolvimento é preciso um *token* pessoal com permissão de leitura de pacotes (PD02).

## 4.4 VERIFICAÇÕES DO CONTRATO

Quadro 10 – Verificações automáticas no repositório do contrato (bloqueiam o PR).

| Verificação | O que garante |
|---|---|
| Spectral (`lint`) | Estilo e completude: `operationId` único, exemplos presentes, descrições, códigos de erro declarados. |

| Verificação | O que garante |
|---|---|
| oasdiff (`breaking`) | Compara com a última *tag* e falha em mudança incompatível, salvo se o PR declara `!` no *commit* e o aumento *major* for compatível com a política (Seção 4.5). |
| Geração Java | O projeto `java/` compila com o gerador configurado. |
| Tipos TypeScript | O pacote `ts/` passa em `tsc --noEmit`. |
| Prism (`mock-smoke`) | O servidor de *mock* sobe e as respostas exemplificadas validam contra seus esquemas. |
| Prettier e `lint` de YAML | Formatação uniforme. |

Fonte: Elaborado pelos autores.

O teste de contrato contra a implementação (Schemathesis) roda no CI do backend, que é onde a aplicação existe (Capítulo 10).

## 4.5 EVOLUÇÃO E COMPATIBILIDADE

- **Antes da 1.0.0:** mudanças incompatíveis são permitidas, com aumento *minor*, e comunicadas no *changelog*.
- **A partir da 1.0.0:** adicionar recurso, campo opcional ou valor de resposta é *minor*; correção de documentação é *patch*; remover ou renomear campo, tornar campo obrigatório ou mudar tipo é *major*.
- **Remoção:** o elemento é marcado `deprecated: true` por ao menos uma versão *minor* antes de ser removido na *major* seguinte; o front deixa de usá-lo antes da remoção.

# 5 BACKEND (`GCC188-BACKEND`)

## 5.1 STACK

Quadro 11 – Stack do backend.

| Componente | Escolha | Observação |
|---|---|---|
| Linguagem | Java 25 (LTS), com *toolchain* do Gradle. | D09. Distribuição Eclipse Temurin no CI e na imagem. |
| *Build* | Gradle 9 com Kotlin DSL, catálogo de versões (`libs.versions.toml`) e *wrapper* versionado. | Spring Boot 4 oferece suporte ao Gradle 9. |
| Framework | Spring Boot 4.x (Spring Framework 7, Jakarta EE 11, Jackson 3). | A linha 4.1 é a estável vigente; a 4.2 está prevista para novembro de 2026. Fixar a versão no marco M0 (PD12). |
| Web | Spring MVC. | Controladores implementam as interfaces geradas do contrato. |
| Segurança | Spring Security 7 com Nimbus JOSE (JWT). | Sem implementação própria de criptografia (Capítulo 7). |
| Persistência | Spring Data JPA, Hibernate ORM, HikariCP e driver PostgreSQL. | `open-in-view` desativado; `ddl-auto=validate`. |
| Migrações | Flyway com o módulo do PostgreSQL. | Seção 6.2. |
| Validação | Jakarta Bean Validation (Hibernate Validator), com `@CPF` e `@CNPJ`. | Valida o formato; unicidade é regra de serviço e restrição de banco. |
| Mapeamento | MapStruct. | D12. Entidade para DTO e DTO para comando. |
| Observabilidade | Actuator, Micrometer e *logs* estruturados em JSON. | D15. |
| Testes | JUnit Jupiter, AssertJ, Mockito, Spring Security Test, Testcontainers (PostgreSQL), ArchUnit e JaCoCo. | D20. |
| Qualidade | Spotless (formatação) e Checkstyle (regras básicas). | Executados no CI e, de forma reduzida, nos ganchos locais. |
| Imagem | `bootBuildImage` (Cloud Native Buildpacks), usuário sem privilégio. | D30. Jib é a alternativa se o tempo de *build* incomodar. |

Fonte: Elaborado pelos autores.

## 5.2 MÓDULOS E CAMADAS

O código é organizado por **módulo de negócio**; dentro de cada módulo, as camadas da seção 9.1 da especificação são mantidas (D10, Δ09). A regra da especificação continua valendo: regras de negócio residem somente na camada de serviço.

Código 4 – Estrutura de pacotes do backend (pacote base proposto, PD03).

```
br.ufla.cooperativa
├── CooperativaApplication
├── shared/            # erros (ProblemDetail), paginação, Clock, auditoria, escopo do usuário
├── auth/              # login, refresh, logout, JWT, CSRF, política de senha
├── usuarios/
├── cooperandos/
├── clientes/
├── contratos/         # máquina de estados, cálculo de taxa
├── pagamentos/        # cronograma, baixa, repasse, estorno
├── despesas/
└── relatorios/        # extrato, resumo, inadimplência, painel (somente leitura)
    └── (em cada módulo) web/  service/  persistence/
```

Fonte: Elaborado pelos autores.

Quadro 12 – Responsabilidade das camadas dentro de um módulo.

| Pacote | Conteúdo |
|---|---|
| `web` | Controladores que implementam as interfaces geradas; mapeadores MapStruct; `@PreAuthorize` por papel. Não contêm regra de negócio e não acessam `persistence`. |
| `service` | Casos de uso, regras RN, transações (`@Transactional`), máquina de estados, publicação de eventos de domínio. Recebe o ator (usuário, papel, vínculo) como parâmetro. |
| `persistence` | Entidades JPA, repositórios Spring Data, `Specification` de filtro e de escopo. |

Fonte: Elaborado pelos autores.

**Comunicação entre módulos** (D41). Para evitar ciclos, dois mecanismos são usados:

- **Eventos de domínio síncronos** (`ApplicationEventPublisher`), executados dentro da transação de quem publica. Exemplo: ao aprovar um contrato, `contratos` publica `ContratoAprovado`; `pagamentos` gera o cronograma na mesma transação (RF-CTR-04, RNF-05). O cancelamento segue o mesmo caminho (RN06).
- **Portas** (interfaces definidas no módulo consumidor e implementadas por outro). Exemplo: o encerramento consulta a porta `PendenciasFinanceiras`, declarada em `contratos` e implementada em `pagamentos` (RN06).

Quadro 13 – Regras de arquitetura verificadas com ArchUnit (falham o CI).

| # | Regra | Origem |
|---|---|---|
| A1 | `web` não depende de `persistence`. | RNF-12 |
| A2 | Classes em `web` e `persistence` não contêm lógica de regra de negócio: somente `service` acessa `Clock` e executa transições de estado. | RNF-12 |
| A3 | Não há ciclos entre módulos. | D10 |
| A4 | Um módulo só usa classes de outro módulo no pacote `service` (tipos públicos) ou eventos. | D41 |
| A5 | É proibido `Instant.now()`, `LocalDate.now()` e `System.currentTimeMillis()` sem `Clock`. | D40 |
| A6 | Entidades JPA não são retornadas por controladores. | RNF-03 |

Fonte: Elaborado pelos autores.

## 5.3 REGRAS DE NEGÓCIO, TRANSAÇÕES E CONCORRÊNCIA

Quadro 14 – Onde cada regra de negócio é aplicada e verificada.

| Regra | Componente | Verificação |
|---|---|---|
| RN02, RN03 | `TaxaCalculadora` em `contratos.service`; `CHECK` no banco. | Teste unitário com tabela de casos (percentual e fixo; taxa maior que o bruto). |

| Regra | Componente | Verificação |
|---|---|---|
| RN05 | `CronogramaPagamentos` em `pagamentos.service`; resíduo de arredondamento na última parcela. | Teste unitário e propriedade "soma das parcelas é igual ao total" (CA-01). |
| RN06, RN07 | `PagamentoService` e `ContratoService`; transições por tabela de estados. | Teste de integração (CA-03, CA-07). |
| RN08 | `DespesaService`; `CHECK` condicional no banco. | Teste de integração (CA-06). |
| RN10 | `EscopoDoUsuario` em `shared`: gera a `Specification` que restringe por cooperando ou cliente. | Teste de integração (CA-04) e teste parametrizado da matriz de permissões. |
| RN11 | Serviços de cadastro: inativação em vez de exclusão; sem método de exclusão em pagamento. | Teste de integração. |
| RN12 | `ContratoEstados`: mapa de transições permitidas por estado e papel; quem elabora não aprova. | Teste unitário exaustivo das transições (CA-05). |
| RN14 | `CooperandoService` e `ClienteService`: criação do usuário e do vínculo em uma transação. | Teste de integração (falha parcial desfaz tudo). |
| RN15 | Serviços de aprovação, baixa e repasse gravam o ator e o instante. | Teste de integração. |

Fonte: Elaborado pelos autores.

**Transações.** Cada método público de serviço que escreve é `@Transactional`; consultas são `@Transactional(readOnly = true)`. Aprovação, cancelamento e cadastro com usuário são atômicos (RNF-05).

**Concorrência otimista** (D13, FOWLER, 2002). Entidades com disputa possível (`Usuario`, `Contrato`, `Pagamento`, `Despesa`) têm coluna `versao` mapeada com `@Version`. Dois casos distintos:

- **Atualização concorrente** (PUT): o DTO carrega `versao`; se difere da gravada, o serviço responde 409 sem alterar o registro.
- **Comando concorrente** (baixa, repasse, aprovação): a segunda execução encontra o estado já alterado e responde 409 pela validação da transição (RN12, RN04); se as duas leem o mesmo estado ao mesmo tempo, a falha de versão do Hibernate é traduzida também em 409.

## 5.4 PERSISTÊNCIA

- **Filtros e escopo** (D11): cada recurso tem uma `Specification` de filtro (parâmetros da consulta) combinada por `and` com a de escopo (RN10). Como a restrição de escopo faz parte da consulta, um registro de outro cooperando simplesmente não é encontrado e a resposta é 404 (RNF-03).
- **Ordenação:** `sort` aceita apenas campos de uma lista permitida por recurso; qualquer outro valor responde 400. O tamanho máximo de página é 100 (RNF-08).
- **Relatórios:** consultas JPQL ou SQL nativo com projeções para DTOs, em transação somente leitura, no módulo `relatorios`.
- **Desempenho:** associações `LAZY`; carga de detalhe com `EntityGraph`; `default_batch_fetch_size` para evitar o problema N+1.

- **Auditoria** (D14): `AuditingEntityListener` preenche `criado_em` e `atualizado_em`; `criado_por_id` do contrato e `registrado_por_id` da despesa usam o `AuditorAware` do usuário autenticado. Aprovação, baixa e repasse gravam explicitamente o ator e o instante (RN15).
- **Identificadores:** `GenerationType.IDENTITY`. Como o volume de inserções por transação é pequeno (no máximo as parcelas de um contrato), a perda de *batch* de inserção é irrelevante.

## 5.5 TRATAMENTO DE ERROS

Um `@RestControllerAdvice` traduz exceções em `ProblemDetail` (RFC 9457) com `application/problem+json`, preenchendo `correlation_id` e, quando aplicável, `campos_invalidos` e `codigo_regra`.

Quadro 15 – Tradução de exceções em respostas HTTP.

| **Situação** | **Status** | **Observação** |
|---|---|---|
| Corpo malformado ou campo com formato inválido (Bean Validation). | 400 | `campos_invalidos` lista cada campo. |
| Regra de negócio violada (`RegraNegocioException`). | 422 | `codigo_regra` (por exemplo `RN03`) e o campo envolvido (CA-02, CA-06). |
| Transição de estado inválida; divergência de versão; unicidade violada. | 409 | Unicidade (e-mail, CPF, matrícula, documento) informa o campo. |
| Não autenticado. | 401 | Sem cabeçalho `WWW-Authenticate`, pois a autenticação é por *cookie*. |
| Papel sem permissão (`AccessDeniedException`). | 403 | Mensagem genérica. |
| Recurso inexistente ou fora do escopo. | 404 | Indistinguível de inexistente (RN10). |
| Falha inesperada. | 500 | Sem detalhes técnicos; o `correlation_id` permite achar o registro no *log*. |

Fonte: Elaborado pelos autores.

## 5.6 TEMPO, MOEDA E ARREDONDAMENTO

- **Instantes** em `Instant` (UTC) e persistidos em `TIMESTAMPTZ`; `hibernate.jdbc.time_zone=UTC`.
- **Datas de negócio** (vencimento, vigência) em `LocalDate`, sem fuso, e trafegadas como texto AAAA-MM-DD.
- **"Hoje"** (D40): obtido de um `Clock` injetado, no fuso `America/Sao_Paulo`; o atraso (P5) compara `data_vencimento` com essa data. Os testes usam relógio fixo.
- **Dinheiro:** `BigDecimal` com escala 2 e `RoundingMode.HALF_UP` (D40); nunca `double`.

## 5.7 OBSERVABILIDADE

- **Actuator** em porta de gerenciamento separada (8081), não roteada pelo proxy. Expostos: `health` (com *probes* de *liveness* e *readiness*), `info` e `metrics`. A porta não é publicada pelo Compose.
- **Logs** em JSON na saída padrão, usando o suporte a *logging* estruturado do Spring Boot. Cada linha inclui `correlation_id`, método, rota (modelo, não o caminho com identificadores), status, duração e `usuario_id`.
- **Correlação:** um filtro lê `X-Request-Id` (gerado pelo proxy ou, na falta, pelo backend), coloca o valor no MDC, devolve-o na resposta e o repete no corpo de erro.
- **Privacidade** (RNF-25): senhas, *tokens*, CPF, CNPJ, e-mail e telefone não são registrados em *log*; corpos de requisição não são registrados.

## 5.8 CONFIGURAÇÃO

O backend segue o fator III dos 12 fatores (WIGGINS, 2017): configuração por ambiente, imagem única. Perfis `dev`, `test` e `prod`; *secrets* lidos de arquivos em `/run/secrets/` por `spring.config.import=optional:configtree:/run/secrets/`.

Quadro 16 – Variáveis e *secrets* principais.

| **Propriedade** | **Finalidade** | **Tipo** |
|---|---|---|
| `spring.datasource.url` | URL JDBC; inclui `sslmode` quando o banco é remoto. | Variável |
| `spring.datasource.username`, `spring.datasource.password` | Credenciais do banco. | *Secret* |
| `app.auth.jwt-secret` | Chave HMAC (256 bits ou mais) do JWT. | *Secret* |
| `app.auth.cookie-secure` | Marca os *cookies* como `Secure`; só é falso no perfil `dev` sem TLS. | Variável |
| `app.bootstrap.admin.email`, `app.bootstrap.admin.password` | Administrador inicial (Seção 6.4). | *Secret* |
| `app.timezone` | Fuso de negócio (`America/Sao_Paulo`). | Variável |

Fonte: Elaborado pelos autores.

# 6 BANCO DE DADOS

## 6.1 SERVIDOR E CONVENÇÕES

O banco é PostgreSQL em contêiner próprio, com volume persistente e a versão *major* fixada em `versions.env` (PD11). Convenções (D18, D19):

- chaves `BIGINT GENERATED ALWAYS AS IDENTITY`; o DER da especificação não muda;
- enumerações como `VARCHAR` com `CHECK`, porque acrescentar valor a `ENUM` nativo exige comandos especiais e remover valor não é suportado;
- instantes em `TIMESTAMPTZ`; datas de negócio em `DATE`;
- valores monetários em `NUMERIC(12,2)` (RNF-04);
- nomes em `snake_case`, que o Spring Boot mapeia por convenção;
- ordenação de nomes com acentuação: avaliar uma *collation* ICU para o português do Brasil, conforme disponibilidade na imagem escolhida (PD14).

## 6.2 MIGRAÇÕES COM FLYWAY

As migrações ficam em `src/main/resources/db/migration` e são aplicadas na inicialização (D17). O Hibernate apenas confere o esquema (`ddl-auto=validate`).

Quadro 17 – Regras para migrações.

| Tema | Regra |
|---|---|
| Nome | `V<AAAAMMDDHHmm>__descricao.sql` (data e hora em UTC), o que evita colisão de números entre ramos paralelos. |
| Imutabilidade | Uma migração integrada em `develop` não é editada; correções entram em nova migração. O Flyway valida somas de verificação ao iniciar. |
| Compatibilidade | Alterações em dois passos (expandir e depois contrair): primeiro acrescentar, depois migrar o código, por fim remover. Permite trocar a versão da aplicação sem quebrar a anterior. |
| Conteúdo | Somente SQL. Dados de domínio fixos não existem (enumerações são `CHECK`). |
| Teste | Toda alteração de esquema é aplicada em PostgreSQL real (Testcontainers) a partir de banco vazio; o CI também aplica as migrações sobre o esquema da última *tag* (teste de atualização). |
| Privilégios | Em produção, recomenda-se um usuário de migração (DDL) distinto do usuário da aplicação (DML). No Compose local, um único usuário é aceitável. |

Fonte: Elaborado pelos autores.

Código 5 – Sequência inicial de migrações (nomes ilustrativos).

```
V202611101000__criar_usuario.sql
V202611101010__criar_cooperando_e_cliente.sql
V202611101020__criar_contrato.sql
V202611101030__criar_pagamento.sql
V202611101040__criar_despesa.sql
V202611101050__criar_refresh_token.sql
V202611101100__criar_indices.sql
```

Fonte: Elaborado pelos autores.

## 6.3 AJUSTES AO MODELO FÍSICO

O modelo da especificação é mantido, com os acréscimos de Δ05 e Δ06. As colunas `versao` (`BIGINT NOT NULL DEFAULT 0`) entram em `usuario`, `contrato`, `pagamento` e `despesa`. A tabela `usuario` recebe `deve_trocar_senha BOOLEAN NOT NULL DEFAULT FALSE`, `falhas_login SMALLINT NOT NULL DEFAULT 0` e `bloqueado_ate TIMESTAMPTZ`. A tabela `refresh_token` é técnica e não faz parte do domínio nem do CRUD exigido pela disciplina.

Código 6 – DDL de `contrato` e `pagamento` com as restrições da especificação.

```
CREATE TABLE contrato (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  cooperando_id     BIGINT        NOT NULL REFERENCES cooperando (id),
  cliente_id        BIGINT        NOT NULL REFERENCES cliente (id),
  descricao         VARCHAR(500)  NOT NULL,
  valor_total       NUMERIC(12,2) NOT NULL CHECK (valor_total > 0),
  modalidade        VARCHAR(10)   NOT NULL CHECK (modalidade IN ('UNICO','MENSAL')),
  num_parcelas      SMALLINT      NOT NULL CHECK (num_parcelas >= 1),
  dia_vencimento    SMALLINT      CHECK (dia_vencimento BETWEEN 1 AND 28),
  data_inicio       DATE          NOT NULL,
  data_fim          DATE,
  taxa_tipo         VARCHAR(10)   NOT NULL CHECK (taxa_tipo IN ('PERCENTUAL','FIXO')),
  taxa_valor        NUMERIC(12,2) NOT NULL CHECK (taxa_valor > 0),
  status            VARCHAR(20)   NOT NULL CHECK (status IN
                      ('RASCUNHO','PENDENTE_APROVACAO','ATIVO','ENCERRADO','CANCELADO')),
  motivo_reprovacao VARCHAR(300),
  criado_por_id     BIGINT        NOT NULL REFERENCES usuario (id),
  aprovado_por_id   BIGINT        REFERENCES usuario (id),
  aprovado_em       TIMESTAMPTZ,
  versao            BIGINT        NOT NULL DEFAULT 0,
  criado_em         TIMESTAMPTZ   NOT NULL DEFAULT now(),
  atualizado_em     TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT ck_contrato_taxa_pct CHECK (taxa_tipo <> 'PERCENTUAL' OR taxa_valor <= 100),
  CONSTRAINT ck_contrato_datas    CHECK (data_fim IS NULL OR data_fim >= data_inicio),
  CONSTRAINT ck_contrato_parcelas CHECK (
    (modalidade = 'UNICO'  AND num_parcelas = 1) OR
    (modalidade = 'MENSAL' AND num_parcelas >= 2 AND dia_vencimento IS NOT NULL))
);

CREATE TABLE pagamento (
  id               BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  contrato_id      BIGINT        NOT NULL REFERENCES contrato (id),
  numero_parcela   SMALLINT      NOT NULL CHECK (numero_parcela >= 1),
  data_vencimento  DATE          NOT NULL,
  valor_bruto      NUMERIC(12,2) NOT NULL CHECK (valor_bruto > 0),
  valor_taxa       NUMERIC(12,2) NOT NULL CHECK (valor_taxa >= 0),
  valor_liquido    NUMERIC(12,2) NOT NULL,
  status           VARCHAR(10)   NOT NULL CHECK (status IN
                     ('PREVISTO','PAGO','REPASSADO','CANCELADO')),
  data_baixa       DATE,
  baixado_por_id   BIGINT        REFERENCES usuario (id),
  data_repasse     DATE,
  repassado_por_id BIGINT        REFERENCES usuario (id),
  observacao       VARCHAR(300),
  versao           BIGINT        NOT NULL DEFAULT 0,
  criado_em        TIMESTAMPTZ   NOT NULL DEFAULT now(),
  atualizado_em    TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_pagamento_parcela UNIQUE (contrato_id, numero_parcela),
  CONSTRAINT ck_pagamento_valores CHECK
    (valor_taxa <= valor_bruto AND valor_liquido = valor_bruto - valor_taxa),
  CONSTRAINT ck_pagamento_baixa   CHECK
    (status NOT IN ('PAGO','REPASSADO') OR data_baixa IS NOT NULL),
  CONSTRAINT ck_pagamento_repasse CHECK
    (status <> 'REPASSADO' OR data_repasse IS NOT NULL)
);
```

Fonte: Elaborado pelos autores.

Código 7 – DDL da tabela técnica `refresh_token` (rotação com detecção de reuso).

```
CREATE TABLE refresh_token (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id  BIGINT      NOT NULL REFERENCES usuario (id),
  token_hash  CHAR(64)    NOT NULL UNIQUE,   -- SHA-256 em hexadecimal; o valor puro nunca é
gravado
  familia     UUID        NOT NULL,          -- cadeia de rotação
  expira_em   TIMESTAMPTZ NOT NULL,
  usado_em    TIMESTAMPTZ,                   -- preenchido ao ser trocado por um novo
  revogado_em TIMESTAMPTZ,
  criado_em   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Fonte: Elaborado pelos autores.

Quadro 18 – Índices (o PostgreSQL não cria índices automaticamente em chaves estrangeiras).

| **Índice** | **Finalidade** |
|---|---|
| `uq_usuario_email` em `lower(email)` | E-mail único sem distinguir maiúsculas (RN14). |
| `ix_contrato_cooperando (cooperando_id, status)` | Escopo do motorista e filtros de listagem. |
| `ix_contrato_cliente (cliente_id, status)` | Escopo do cliente e filtros de listagem. |
| `ix_pagamento_status_venc (status, data_vencimento)` | Atraso, painel e relatórios de inadimplência (RF-PAG-05, RF--REL-03). |
| `ix_despesa_coop_data (cooperando_id, data_despesa)` | Extrato do cooperando e escopo do motorista. |
| `ix_despesa_origem_cat_data (origem, categoria, data_despesa)` | Totais por categoria e período (RF--DSP-05). |
| `ix_refresh_usuario (usuario_id)` | Revogação de todos os *tokens* de um usuário. |

Fonte: Elaborado pelos autores.

## 6.4 ADMINISTRADOR INICIAL

A senha do primeiro administrador não pode estar em uma migração, pois o repositório é versionado. Por isso um `ApplicationRunner` cria o administrador no primeiro *boot* (D17):

- executa somente se não existir usuário `ADMINISTRADOR` e se `app.bootstrap.admin.email` e `app.bootstrap.admin.password` estiverem definidas;
- grava o usuário com `deve_trocar_senha = true`, de modo que a senha de partida é trocada no primeiro acesso;
- registra em *log* apenas que o administrador foi criado, sem o e-mail nem a senha;
- é idempotente: nas inicializações seguintes, não faz nada.

## 6.5 BACKUP E RETENÇÃO

Quadro 19 – Política de backup e de retenção (D37).

| Item | Política |
|---|---|
| Backup lógico | `pg_dump` no formato customizado, uma vez por dia às 02:00 (fuso `America/Sao_Paulo`), por um contêiner dedicado. |
| Retenção dos backups | 7 diários, 4 semanais e 6 mensais, com remoção automática. |
| Cópia externa | Em produção, cópia criptografada para fora do *host* (armazenamento de objetos ou unidade remota). Local: opcional. |
| Teste de restauração | O *script* `restore-check` restaura o dump mais recente em um PostgreSQL descartável e confere contagens e a soma dos pagamentos por contrato (RN05). Roda no CI do repositório de infraestrutura e, na operação, semanalmente e antes de cada release. |
| Objetivos | RPO de 24 horas e RTO de 4 horas (RNF-22). Recuperação em ponto no tempo (arquivamento de WAL) fica como evolução para a nuvem. |
| Logs da aplicação e do proxy | Retenção operacional alvo de 30 dias, obtida por rotação por tamanho no driver `local` do Docker (`max-size` e `max-file` dimensionados para o volume previsto). No perfil `observability`, a retenção é definida em tempo (720 h) no Loki. |
| Dados de auditoria | Ficam no banco (RN11, RN15) e não são expurgados enquanto o sistema estiver em uso. |
| Dados pessoais (LGPD) | Os backups contêm CPF, CNPJ, e-mail e telefone: são criptografados fora do *host*, com acesso restrito. Os *logs* não contêm dados pessoais (Seção 5.7). |

Fonte: Elaborado pelos autores.

# 7 AUTENTICAÇÃO E AUTORIZAÇÃO

## 7.1 VISÃO GERAL

A autenticação é própria, com Spring Security (D21), e reaproveita bibliotecas consolidadas para tudo que é criptográfico: JWT pelo Nimbus JOSE e *hash* de senha pelo `DelegatingPasswordEncoder`. Os *tokens* viajam em *cookies* que o JavaScript do navegador não consegue ler, o que reduz o efeito de uma falha de XSS (OWASP, [s. d.]).

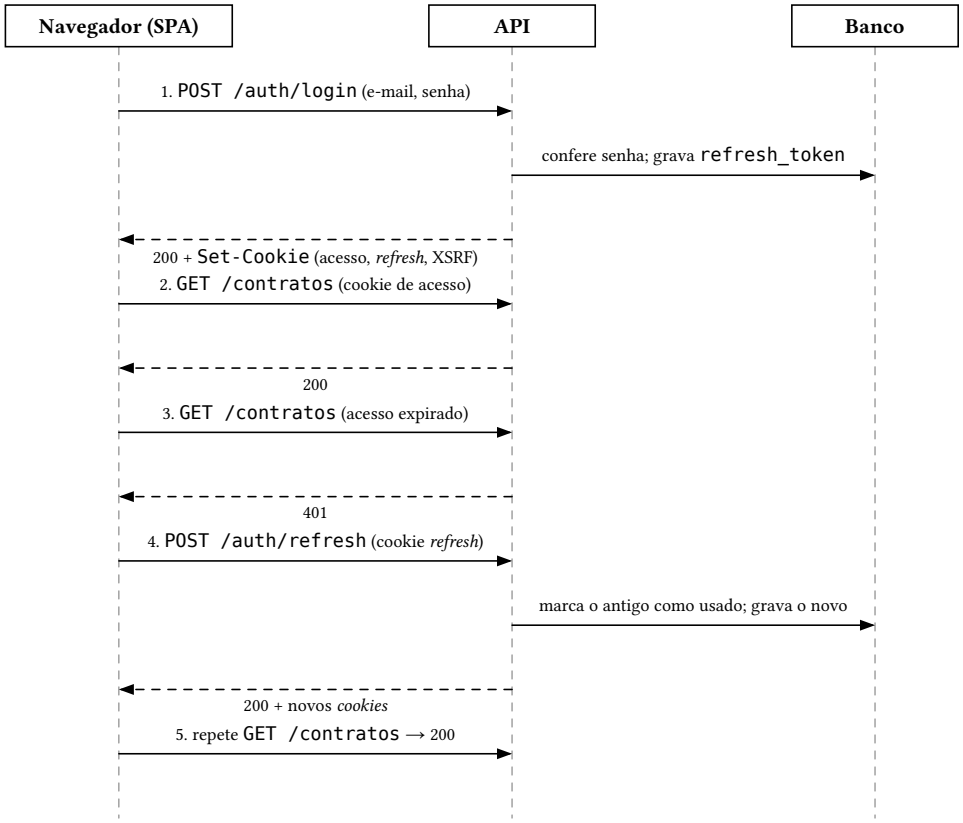
Quadro 20 – Credenciais e *cookies*.

| Item | Característica | Atributos do *cookie* |
|---|---|---|
| `ACCESS_TOKEN` | JWT assinado com HMAC-SHA256, validade de 15 minutos. *Claims*: `sub` (id do usuário), `papel`, `vinculo` (id do cooperando ou do cliente), `iat`, `exp` e `jti`. | `HttpOnly; Secure; SameSite=Strict; Path=/api` |
| `REFRESH_TOKEN` | Valor aleatório de 256 bits; no banco fica apenas o SHA-256. Validade de 7 dias, rotativo. | `HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth` |
| `XSRF-TOKEN` | Valor aleatório para proteção CSRF; o front o lê e o devolve no cabeçalho `X-XSRF-TOKEN`. | `Secure; SameSite=Strict; Path=/` (legível pelo JavaScript) |

Fonte: Elaborado pelos autores.

O prefixo `__Host-` não é usado porque exige `Path=/`, o que impediria restringir o caminho dos *cookies* de *token*. Como o front e a API compartilham a origem (D28), não há CORS e `SameSite=Strict` funciona sem exceções.

Figura 4 – Sequência de login, expiração do *token* de acesso e renovação. A SPA renova uma única vez por falha e, se a renovação falhar, volta ao *login* (CA-10).



Fonte: Elaborado pelos autores.

## 7.2 ROTAÇÃO DO *REFRESH TOKEN* E REVOGAÇÃO

- A cada renovação, o *token* usado recebe `usado_em` e um novo *token* da mesma `familia` é emitido.
- Se um *token* já usado reaparece (indício de roubo), **toda a família** é revogada e o usuário precisa entrar de novo.
- `POST /auth/logout`, troca de senha, inativação do usuário e redefinição pelo administrador revogam os *tokens* de renovação do usuário.
- Um usuário inativado perde o acesso em até 15 minutos (validade do *token* de acesso); a renovação é negada de imediato.

## 7.3 CSRF E CABEÇALHOS DE SEGURANÇA

Além de `SameSite=Strict`, requisições `POST`, `PUT`, `PATCH` e `DELETE` exigem o cabeçalho `X-XSRF-TOKEN` igual ao *cookie* `XSRF-TOKEN` (padrão *double submit* do Spring Security para SPAs; OWASP, [s. d.]). A ausência responde 403 (CA-11). O proxy acrescenta HSTS, `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy` e `frame-ancestors 'none'` (Seção 9.3).

## 7.4 SENHAS E CONTAS

Quadro 21 – Política de senhas e de contas (a validar, PD04 e PD05).

| **Tema** | **Política** |
|---|---|
| Armazenamento | *Hash* `bcrypt` com custo 12 pelo `DelegatingPasswordEncoder`, que grava o prefixo do algoritmo e permite migrar para Argon2 sem perder as senhas existentes (RNF-01). |
| Tamanho | Mínimo de 12 e máximo de 72 bytes (limite do `bcrypt`); sem regras de composição obrigatórias e com bloqueio de senhas comuns. A orientação do NIST (NIST, 2025) privilegia o tamanho em vez da composição e indica 15 caracteres como mínimo para senha usada isoladamente; a equipe pode adotar esse valor (PD04). |
| Bloqueio | Após 5 falhas consecutivas, a conta fica bloqueada por 15 minutos (`falhas_login`, `bloqueado_ate`). A resposta ao *login* é a mesma para e-mail inexistente, senha errada e conta bloqueada. |
| Primeiro acesso | Usuário criado pelo administrador, ou com senha redefinida por ele, recebe `deve_trocar_senha = true`; a API só aceita a troca de senha e o logout até que isso ocorra. |
| Redefinição | Pelo administrador, que informa uma senha temporária (`POST /usuarios/{id}/redefinicao-senha`). Não há envio de e-mail (D22). |

Fonte: Elaborado pelos autores.

## 7.5 AUTORIZAÇÃO

A autorização tem duas camadas, que juntas implementam a matriz da seção 11 da especificação:

a) **Papel** (RBAC): `@PreAuthorize` em cada método do controlador, com `@EnableMethodSecurity`. Papel insuficiente responde 403.
b) **Propriedade do recurso** (RN10): a consulta do serviço recebe a restrição de escopo do usuário (`EscopoDoUsuario`). Registro de outro usuário não é encontrado e responde 404.

A matriz é testada de forma exaustiva por um teste parametrizado: para cada par (operação, papel), compara-se o resultado esperado (2xx, 403 ou 404) com o da API, usando um arquivo CSV derivado da matriz como fonte do teste.

## 7.6 RECUPERAÇÃO DE SENHA POR E-MAIL

O stakeholder perguntou se configurar SMTP seria muito mais complicado. O código em si é pequeno (o Spring Boot oferece `JavaMailSender`). O custo real está em três pontos: o fluxo seguro de redefinição (*token* de uso único com expiração, resposta que não revela se o e-mail existe, invalidação de sessões); a **entregabilidade** em produção (provedor SMTP, e autenticação do domínio com SPF, DKIM e DMARC, sem os quais as mensagens caem em *spam*); e a gestão de mais um *secret*. Para desenvolvimento, um contêiner de captura de e-mails (Mailpit) elimina a dependência externa. Como o escopo da especificação exclui integrações externas, a v1 fica sem e-mail, e o design reserva a porta `NotificadorDeUsuario` no módulo `auth` para uma implementação futura (PD10).

# 8 FRONT-END (GCC188-FRONT)

## 8.1 STACK

O PHP foi considerado, mas descartado: o backend em Java já concentra regras e autenticação, de modo que um servidor PHP seria apenas uma camada intermediária sem função, contrariando a premissa P6 (SPA que acessa a API diretamente). A escolha (D23, D24) prioriza facilidade de desenvolvimento e de prototipagem.

Quadro 22 – Stack do front-end.

| Componente | Escolha | Observação |
|---|---|---|
| Linguagem e *build* | TypeScript em modo estrito; Vite. | Node LTS; npm com `package-lock.json` versionado e `npm ci` no CI. |
| Biblioteca de interface | React 19.2 ou superior. | Requisito do Mantine 9. |
| Componentes | Mantine 9 (`core`, `hooks`, `dates`, `notifications`, `modals`, `charts`). | Formulários, tabelas, diálogos de confirmação (RF-WEB-10) e gráficos do painel (RF--WEB-08) prontos. |
| Tabelas de dados | `mantine-datatable` ou TanStack Table sobre o `Table` do Mantine. | Confirmar a compatibilidade com o Mantine 9 no marco M2 (PD06). |
| Rotas | React Router. | Rotas protegidas por papel. |
| Dados do servidor | TanStack Query. | *Cache*, estados de carregamento e erro (RNF-17). |
| Formulários e validação | React Hook Form e Zod (`@hookform/resolvers`). | Esquemas Zod gerados do contrato (RF--WEB-04). |
| Cliente da API e *mocks* | Orval (gera *hooks*, esquemas Zod e *handlers* MSW). | Entrada: documento do pacote `@ORG/api-contract`. |
| Formatação | `Intl` e `dayjs` em `pt-BR`. | RNF-21. |
| Testes | Vitest, Testing Library, MSW e `vitest-axe`. | D26. |
| Qualidade | ESLint (`typescript-eslint`, `react-hooks`, `jsx-a11y`) e Prettier. | Executados no CI e nos ganchos locais. |

Fonte: Elaborado pelos autores.

## 8.2 ORGANIZAÇÃO DO CÓDIGO

Código 8 – Estrutura de diretórios do front-end.

```
src/
├── app/              # provedores, roteador, layout (AppShell), tema do Mantine
├── features/         # um diretório por módulo
│   ├── auth/  usuarios/  cooperandos/  clientes/
│   ├── contratos/  pagamentos/  despesas/  relatorios/  painel/
│   └── (em cada um) pages/  components/  forms/  hooks/
├── shared/
│   ├── api/          # fetch com tratamento de 401, 403, 409 e 422
│   ├── generated/    # saída do Orval (não versionada)
│   ├── permissions/  # matriz de permissões (espelho da seção de permissões)
│   ├── ui/           # ListPage, FormPage, DetailPage, ConfirmButton, Dinheiro, Data
│   └── lib/          # formatadores pt-BR
├── mocks/            # handlers MSW e cenários
└── main.tsx
```

Fonte: Elaborado pelos autores.

## 8.3 DADOS E CONTRATO

O Orval lê o documento OpenAPI do pacote npm e gera, a cada `npm run generate` (executado antes do `dev`, dos testes e do *build*), os *hooks*, os esquemas Zod e os *handlers* MSW em `src/shared/generated/`, diretório que não é versionado. Assim, uma mudança no contrato que quebra o front aparece como erro de compilação, e não como falha em tempo de execução.

O cliente HTTP (`shared/api`) padroniza o tratamento de respostas:

- **401:** tenta uma única renovação (`POST /auth/refresh`), com uma só chamada simultânea compartilhada entre as requisições pendentes; se falhar, limpa a sessão e redireciona ao *login* (RF-WEB-01, CA-10);
- **400 e 422:** converte `campos_invalidos` em erros do React Hook Form, exibidos junto ao campo (RF-WEB-04, CA-09);
- **403 e 409:** notificação com a mensagem do problema; no 409, os dados são recarregados;
- **escrita:** lê o *cookie* `XSRF-TOKEN` e o envia em `X-XSRF-TOKEN`.

Regras de formatação: valores monetários exibidos com `Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' })`, sem aritmética no cliente (D39); datas de vencimento mantidas como texto AAAA-MM-DD e formatadas por `dayjs`, nunca convertidas por `new Date('AAAA-MM-DD')`, que interpreta em UTC e pode mostrar o dia anterior.

## 8.4 SESSÃO E PERMISSÕES NA INTERFACE

Como o *token* não é legível pelo JavaScript, a SPA obtém usuário, papel e vínculo por `GET /auth/me` ao iniciar e guarda o resultado no contexto de sessão. O módulo `shared/permissions` contém uma tabela `ação → papéis` que espelha a matriz de permissões; menus, rotas e botões consultam essa tabela (RF-WEB-02, RF-WEB-11). A função pura

`acoesDisponiveis(contrato, papel)` calcula as transições visíveis no detalhe do contrato (RF-WEB-05, RN12) e tem teste exaustivo (CA-08). A API continua sendo a autoridade (RNF-03).

## 8.5 TELAS PADRONIZADAS

Para reduzir o esforço (R7), as telas reutilizam três componentes:

- `ListPage`: filtros, ordenação e paginação mantidos na *query string* (a lista filtrada pode ser compartilhada por link), estados de carregamento, vazio e erro (RF-WEB-03, RNF-17);
- `FormPage`: formulário de criação e edição com validação Zod e erros do servidor (RF--WEB-04);
- `DetailPage`: detalhe com ações condicionadas ao papel e ao estado, e diálogo de confirmação para ações irreversíveis (RF-WEB-05, RF-WEB-10).

## 8.6 PROTOTIPAGEM SEM BACKEND

- `npm run dev:mock` sobe o Vite com MSW; as respostas vêm dos exemplos do contrato e de cenários em `src/mocks/` (por exemplo, contrato pendente de aprovação, pagamento em atraso, erro 422). Permite construir e revisar uma tela antes de o backend existir (D25).
- `npm run dev` usa o proxy do Vite (`/api` para o backend local), reproduzindo a mesma origem da produção. No perfil `dev` sem TLS, o backend define `app.auth.cookie-secure=false`.
- O Prism, executado a partir do repositório do contrato, serve o mesmo documento como *mock* HTTP para ferramentas externas e testes manuais.

## 8.7 ENTREGA

A imagem do front é construída em dois estágios: um estágio Node executa `npm ci`, `npm run generate` e `npm run build`; o estágio final usa Nginx sem privilégios apenas para servir os arquivos, com retorno a `index.html` nas rotas da SPA, cache longo para `/assets/` (nomes com *hash*), sem cache para `index.html` e uma rota `/healthz`. Como o front acessa `/api` na mesma origem, a imagem não precisa de configuração em tempo de execução. O acesso ao pacote npm privado durante o *build* usa um *secret* do BuildKit, que não permanece na imagem.

# 9 INFRAESTRUTURA E IMPLANTAÇÃO (`GCC188-INFRA`)

## 9.1 ESTRUTURA DO REPOSITÓRIO

Código 9 – Estrutura de diretórios do repositório de infraestrutura.

```
gcc188-infra/
├── compose.yaml            # serviços base: proxy, front, backend, db, backup
├── compose.dev.yaml        # sobreposição local (portas em 127.0.0.1, certificado mkcert)
├── compose.prod.yaml       # sobreposição de produção (domínio, ACME, limites, logs)
├── versions.env            # versões das imagens: backend, front, PostgreSQL, Caddy
├── .env.example            # variáveis não secretas; .env real fora do Git
├── caddy/                  # Caddyfile (produção) e Caddyfile.dev
├── backup/                 # imagem e scripts: backup.sh, restore-check.sh
├── e2e/                    # testes Playwright do sistema integrado
├── observability/          # perfil opcional: Loki, Alloy e Grafana
├── scripts/                # init-env.sh, mkcert.sh, up.sh, reset-dev.sh
└── docs/runbook.md         # operação: subir, atualizar, restaurar, rotacionar segredos
```

Fonte: Elaborado pelos autores.

## 9.2 SERVIÇOS DO COMPOSE

Quadro 23 – Serviços, redes e verificações de saúde.

| Serviço | Imagem | Rede | Saúde e observações |
|---|---|---|---|
| `proxy` | `caddy:<versão>` (fixada em `versions.env`). | `edge` | Únicos portos publicados (80 e 443). TLS, cabeçalhos de segurança, roteamento e *logs* de acesso em JSON. |
| `front` | `ghcr.io/ORG/gcc188-front:${FRONT_VERSION}` | `edge` | `GET /healthz`. Sem privilégios, sistema de arquivos somente leitura. |
| `backend` | `ghcr.io/ORG/gcc188-backend:${BACKEND_VERSION}` | `edge`, `internal` | *Readiness* em `:8081/actuator/health/readiness` (porta de gerenciamento não publicada). *Secrets* por arquivos. Só inicia depois de o banco ficar saudável. |
| `db` | `postgres:${POSTGRES_MAJOR}` | `internal` | `pg_isready`. Volume `pgdata`. Porta não publicada em produção; em desenvolvimento, vinculada a 127.0.0.1. |
| `backup` | Imagem própria mínima com cliente do PostgreSQL e agendador. | `internal` | Executa `backup.sh` diariamente; volume `backups`. |

Fonte: Elaborado pelos autores.

A rede `internal` é declarada com `internal: true` (sem saída para fora do *host*), e somente `proxy`, `front` e `backend` participam de `edge`. Endurecimento aplicado a todos os serviços: `restart: unless-stopped`, `cap_drop: [ALL]`, `security_opt: [no-new-privileges:true]`, limites de memória e CPU e *logs* com rotação.

## 9.3 ROTEAMENTO, CABEÇALHOS E TLS

Código 10 – Trecho do `Caddyfile` de produção (D28, D29).

```
{$APP_DOMAIN} {
  encode zstd gzip
  header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    Referrer-Policy "strict-origin-when-cross-origin"
    Content-Security-Policy "default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
data:; base-uri 'self'; form-action 'self'; frame-ancestors 'none'"
    -Server
  }
  handle /api/* {
    reverse_proxy backend:8080
  }
  handle {
    reverse_proxy front:8080
  }
  log {
    output stdout
    format json
  }
}
```

Fonte: Elaborado pelos autores.

O estilo `'unsafe-inline'` é necessário apenas para estilos, porque o React aplica estilos por atributo; *scripts* continuam restritos à própria origem. O HSTS não entra no `Caddyfile.dev`, pois o navegador o guardaria para `localhost`.

Quadro 24 – TLS por cenário (D29).

| **Trecho** | **Desenvolvimento local** | **Produção (*host* ou nuvem)** |
|---|---|---|
| Navegador para proxy | Certificado emitido pelo mkcert para `localhost`, com a autoridade local instalada no sistema; montado no Caddy por `Caddyfile.dev`. | Let's Encrypt via ACME automático do Caddy, com renovação automática. Exige domínio apontado para o *host* e as portas 80 e 443 abertas. O volume `caddy_data` é persistente. |
| Proxy para backend e front | HTTP na rede privada `edge`. | HTTP na rede privada de um único *host*. Se proxy e aplicação ficarem em *hosts* diferentes, usar TLS interno ou mTLS. |
| Backend para PostgreSQL | `sslmode=disable` na rede `internal`. | `sslmode=disable` no mesmo *host*; `verify-full` com certificado da autoridade quando o banco for gerenciado ou remoto (apenas a URL JDBC muda). |
| Versões do protocolo | TLS 1.2 ou superior. | TLS 1.2 ou superior (padrão do Caddy). |

Fonte: Elaborado pelos autores.

## 9.4 SEGREDOS E CONFIGURAÇÃO

- **Desenvolvimento:** `scripts/init-env.sh` gera `.env` e os arquivos de *secrets* com valores aleatórios (`openssl rand`); ambos estão no `.gitignore`.
- **Produção:** arquivos de *secrets* com permissão 0400, montados pelo mecanismo `secrets` do Compose em `/run/secrets/` (D31). O backend os lê por `configtree`.

- **CI:** GitHub Secrets e Variables; nenhum *secret* de aplicação é necessário no CI, que usa apenas o `GITHUB_TOKEN`.
- **Rotação:** a troca da chave do JWT invalida as sessões ativas, o que é aceitável; o procedimento está no *runbook*.

## 9.5 CAMINHO PARA A NUVEM

A v1 roda somente em Compose (D27), mas nada nela prende o sistema a um *host* específico:

a) **Mesma pilha em uma VPS:** `docker compose -f compose.yaml -f compose.prod.yaml up -d`, com `APP_DOMAIN` e os *secrets* do servidor.
b) **Banco gerenciado:** os serviços `db` e `backup` pertencem ao perfil `local-db` (ativado em `COMPOSE_PROFILES`); desativá-lo e informar a URL JDBC com `sslmode=verify-full` basta. O backup passa a ser do provedor.
c) **Orquestradores** (Cloud Run, ECS, Kubernetes): as imagens são OCI, a configuração é por variáveis e *secrets*, e o backend é sem estado (sessão em JWT e *refresh token* no banco; bloqueio de login no banco), de modo que escala horizontalmente com a mesma chave de JWT.

## 9.6 OBSERVABILIDADE OPCIONAL

O perfil `observability` acrescenta Loki, Grafana Alloy e Grafana, com retenção de 720 horas, para consultar os *logs* JSON por `correlation_id`. Não é necessário para a v1 e está desativado por padrão.

# 10 INTEGRAÇÃO CONTÍNUA

## 10.1 GANCHOS LOCAIS

Os ganchos são uma conveniência para falhar cedo; o CI executa as mesmas verificações e é a garantia (D33). A ferramenta é o lefthook (binário único, independente de linguagem), com `lefthook.yml` versionado em cada repositório.

Quadro 25 – Ganchos locais por repositório.

| **Repositório** | **pre-commit** | **commit-msg** | **pre-push** |
|---|---|---|---|
| `gcc188-backend` | `spotlessApply` nos arquivos alterados. | commitlint (*Conventional Commits*). | `./gradlew test` (testes unitários). |
| `gcc188-front` | Prettier e ESLint com correção nos arquivos alterados. | commitlint. | `tsc --noEmit` e `vitest run`. |
| `gcc188-api-contract` | Spectral e Prettier nos YAML alterados. | commitlint. | Verificação de geração Java e TypeScript. |
| `gcc188-infra` | `yamllint`, `hadolint`, `shellcheck` e `docker compose config -q`. | commitlint. | — |

Fonte: Elaborado pelos autores.

## 10.2 PIPELINES POR REPOSITÓRIO

Todos os *workflows* ficam em `.github/workflows/` e usam GitHub Actions (D32). As verificações abaixo são obrigatórias para integrar PR (D34).

Quadro 26 – Verificações dos repositórios `gcc188-backend` e `gcc188-front`.

| **Verificação** | **Backend** | **Front** |
|---|---|---|
| Estilo e tipos | `spotlessCheck` e Checkstyle. | ESLint, Prettier (`--check`) e `tsc --noEmit`. |
| Testes | Unitários e de integração com Testcontainers; JaCoCo com limite mínimo (RNF-11); regras ArchUnit. | Vitest com cobertura; verificação de acessibilidade com `vitest-axe`. |
| Contrato | Sobe a aplicação e executa o Schemathesis contra o documento do contrato (testa cada recurso com entradas geradas). | Geração do cliente a partir do pacote do contrato e compilação. |
| *Build* | `bootBuildImage` (sem publicar). | `npm run build` com orçamento de tamanho do pacote inicial. |
| Desempenho e acessibilidade | — | Lighthouse CI sobre o *build* servido (orçamentos em Seção 12.3). |
| Segurança | CodeQL (Java), revisão de dependências no PR e varredura da imagem (Trivy). | CodeQL (JavaScript e TypeScript), `npm audit` (dependências de produção), revisão de dependências no PR e varredura da imagem (Trivy). |

| Verificação | Backend | Front |
|---|---|---|
| Governança | commitlint no título do PR e nos *commits*; `source-branch-guard` para PRs em `master`. | Idem. |
| Migrações | Aplicação das migrações em banco vazio e sobre o esquema da última *tag*. | — |

Fonte: Elaborado pelos autores.

Quadro 27 – Verificações dos repositórios `gcc188-api-contract` e `gcc188-infra`.

| Repositório | Verificações |
|---|---|
| `gcc188-api-contract` | Spectral, oasdiff contra a última *tag*, compilação do projeto Java gerado, `tsc` do pacote TypeScript, `mock-smoke` com Prism, Prettier e commitlint (Seção 4.4). |
| `gcc188-infra` | `yamllint`, `hadolint`, `shellcheck`, `docker compose config`, Trivy na configuração, **teste ponta a ponta** (sobe o Compose com as versões de `versions.env` e executa o Playwright: fluxo de login, aprovação de contrato, baixa e repasse, CA-08 a CA-10) e **teste de backup** (gera dados, executa `backup.sh` e valida com `restore-check.sh`). |

Fonte: Elaborado pelos autores.

Os testes ponta a ponta ficam no repositório de infraestrutura porque validam o conjunto, e é ali que o Compose do sistema completo existe. Eles usam as imagens `edge` mais recentes de `develop` em execuções agendadas e as versões fixadas em `versions.env` nos PRs.

## 10.3 PUBLICAÇÃO DE IMAGENS E PACOTES

- **Em *push* para `develop`:** o backend e o front publicam imagens `edge` e `sha-<commit>` no GHCR, para alimentar os testes ponta a ponta. Não há implantação (D35).
- **Em *release* publicada:** publicação das imagens com as etiquetas `X.Y.Z` e `X.Y`, além dos rótulos OCI (`org.opencontainers.image.source`, `version` e `revision`), que ligam a imagem ao repositório. O contrato publica os pacotes Maven e npm com a mesma versão.
- **Atualização de versões:** o Dependabot abre PRs para Gradle, npm, GitHub Actions, Docker e Docker Compose. As versões das imagens em `versions.env` avançam por PR no repositório de infraestrutura, que só passa se o teste ponta a ponta passar.
- **Cadeia de suprimentos:** varredura de vulnerabilidades da imagem e da árvore de dependências; atestado de proveniência da imagem é recomendável e opcional.

# 11 SEGURANÇA: VISÃO CONSOLIDADA

Quadro 28 – Ameaças e controles (categorias do OWASP ASVS, OWASP, 2025).

| **Ameaça** | **Controle** | **Onde** |
|---|---|---|
| Credenciais fracas ou roubadas | `bcrypt` custo 12, política de tamanho, bloqueio temporário, mensagem única de falha. | `auth`, Capítulo 7 |
| Roubo de *token* por XSS | *Cookies* `HttpOnly`; CSP restritiva; *escape* padrão do React; sem `dangerouslySetInnerHTML`. | Proxy, front |
| CSRF | `SameSite=Strict` e *double submit* com `X-XSRF-TOKEN`. | `auth`, front |
| Reuso de *refresh token* roubado | Rotação com detecção de reuso e revogação da família. | `auth`, banco |
| Acesso indevido a dados de outro usuário | RBAC por método e escopo na consulta (404); teste exaustivo da matriz. | Backend |
| Injeção de SQL | JPA parametrizado; `sort` com lista permitida; nenhuma concatenação de SQL. | Backend |
| Vazamento em respostas e *logs* | Entidades nunca expostas; erro 500 sem detalhes; *logs* sem dados pessoais. | Backend |
| Interceptação | TLS no proxy; HSTS; `Secure` nos *cookies*. | Proxy |
| Dependência ou imagem vulnerável | Dependabot, CodeQL, Trivy e revisão de dependências bloqueando PR. | CI |
| Segredo no repositório | Segredos fora do Git; arquivos 0400; varredura de segredos do GitHub ativada. | Infra, CI |
| Perda de dados | Backup diário, cópia externa e teste de restauração. | Infra |

Fonte: Elaborado pelos autores.

# 12 TESTES E REQUISITOS NÃO FUNCIONAIS REVISADOS

## 12.1 ESTRATÉGIA DE TESTES

Quadro 29 – Níveis de teste.

| Nível | Escopo | Ferramentas e local |
|---|---|---|
| Unitário | Regras puras: taxa, cronograma, estados, `acoesDisponiveis`, formatadores. | JUnit e AssertJ (backend); Vitest (front). |
| Fatia (*slice*) | Controlador com segurança (`@WebMvcTest`); repositórios e `Specification` (`@DataJpaTest` com PostgreSQL real). | Spring Test e Testcontainers. |
| Integração | Aplicação completa com banco real; CA-01 a CA-07, CA-11, CA-12 e CA-14; matriz de permissões. | `@SpringBootTest`, Testcontainers. |
| Contrato | A implementação respeita o documento (formatos, códigos, esquemas). | Schemathesis no CI do backend. |
| Componente (front) | Telas com API simulada: CA-08 a CA-10, erros do servidor em campos, acessibilidade. | Vitest, Testing Library, MSW e `vitest-axe`. |
| Ponta a ponta | Sistema integrado em Compose: fluxo principal e CA-08 a CA-10. | Playwright, no repositório de infraestrutura. |
| Não funcional | Desempenho (k6) e orçamentos do front (Lighthouse CI). | k6 em `gcc188-infra/e2e`; Lighthouse CI no front. |

Fonte: Elaborado pelos autores.

Os testes automatizados cobrem cada RN pelo menos uma vez (RNF-11). Testes que dependem de data usam um `Clock` fixo.

## 12.2 CRITÉRIOS DE ACEITAÇÃO ADICIONAIS

Os cenários CA-01 a CA-10 da especificação são mantidos. O design acrescenta:

Quadro 30 – Cenários de aceitação adicionais.

| ID | Dado | Quando | Então |
|---|---|---|---|
| CA-11 | Usuário autenticado com *cookies* válidos. | Envia `POST` sem o cabeçalho `X-XSRF-TOKEN`. | `403`; nenhum dado é alterado (Capítulo 7). |
| CA-12 | *Refresh token* já utilizado em uma renovação. | O mesmo valor é apresentado de novo. | `401` e revogação de toda a família do *token*. |
| CA-13 | Banco com dados de teste e um backup recente. | `restore-check` restaura o backup em banco descartável. | Contagens e somas dos pagamentos conferem com a origem (Seção 6.5). |
| CA-14 | Banco vazio, e banco no esquema da última *tag*. | As migrações são aplicadas. | Concluem sem erro e o Hibernate valida o esquema (Seção 6.2). |
| CA-15 | Usuário criado pelo administrador com `deve_trocar_senha`. | Tenta listar contratos antes de trocar a senha. | `403` com `codigo_regra` indicando a troca; após a troca, o acesso é liberado (Capítulo 7). |

Fonte: Elaborado pelos autores.

## 12.3 REQUISITOS NÃO FUNCIONAIS REVISADOS E NOVOS

Os valores abaixo substituem os originais onde indicado (D36). As metas de desempenho foram calibradas para o porte real do sistema, uma cooperativa com milhares de registros, e para um ambiente de referência modesto.

Quadro 31 – RNF revisados (Δ08) e novos.

| ID | Requisito revisado ou novo | Valor anterior | Verificação |
|---|---|---|---|
| RNF-02 | *Token* de acesso com validade de 15 min; *refresh token* de 7 dias, rotativo; HTTPS na implantação. | Validade máxima de 60 min. | Teste de integração |
| RNF-10 | No ambiente de referência (backend com 1 CPU e 1 GiB, PostgreSQL com 1 GiB, 10 usuários virtuais) e com massa de 1.000 contratos, 12.000 pagamentos e 5.000 despesas: listagens com p95 de até 800 ms; comandos (aprovar, baixa, repasse) com p95 de até 1 s; relatórios com p95 de até 2 s. | p95 ≤ 500 ms para até 10 mil registros em ambiente local. | k6, antes de cada *release* |
| RNF-11 | Cobertura de linhas de pelo menos 70% na camada de serviço (limite no JaCoCo) e toda RN com teste. No front, pelo menos 60% das linhas em `shared/lib`, `shared/permissions` e `hooks`, excluindo componentes puramente visuais. | Cobertura de 70% da camada de serviço. | JaCoCo e Vitest no CI |
| RNF-12 | Pacotes por módulo com camadas internas; regras A1 a A6 verificadas por ArchUnit. | Arquitetura em camadas, por revisão. | ArchUnit |
| RNF-15 | O ambiente completo sobe com um comando a partir do repositório de infraestrutura, em máquina com Docker, sem instalar Java ou Node. | Execução reproduzível via README. | Teste ponta a ponta |
| RNF-19 | *Cookies* `HttpOnly`, `Secure` e `SameSite=Strict`; proteção CSRF; CSP; mesma origem, sem CORS habilitado. | CORS restrito à origem do front. | Teste e revisão |
| RNF-20 | Lighthouse CI em predefinição de computador sobre o *build* servido: LCP de até 2,5 s, pontuação de desempenho de pelo menos 80 e JavaScript inicial de até 350 kB compactado. | Tela inicial em até 3 s em rede local. | Lighthouse CI |
| RNF-18 | Complemento: pontuação de acessibilidade do Lighthouse de pelo menos 90 e nenhuma violação grave do `axe` nas telas cobertas por teste. | Meta de WCAG 2.1 AA, sem medida. | `vitest-axe`, Lighthouse CI |
| RNF-22 | Disponibilidade e recuperação: RPO de 24 h e RTO de 4 h; *backup* diário; restauração testada (CA-13). | Novo. | `restore-check` |
| RNF-23 | Observabilidade: *logs* JSON com `correlation_id`; *probes* de saúde; retenção operacional de 30 dias. | Novo. | Inspeção |

| ID | Requisito revisado ou novo | Valor anterior | Verificação |
|---|---|---|---|
| RNF-24 | Cadeia de suprimentos: nenhuma vulnerabilidade crítica ou alta sem exceção documentada em imagens e dependências; versões e imagens fixadas. | Novo. | Trivy, Dependabot |
| RNF-25 | Privacidade (LGPD): dados pessoais ausentes dos *logs*; *backups* externos criptografados. | Novo. | Revisão e teste |
| RNF-26 | Reprodutibilidade: Gradle *wrapper*, `package-lock.json` e etiquetas de imagem versionados. | Novo. | Inspeção |

Fonte: Elaborado pelos autores.

# 13 PLANO DE IMPLEMENTAÇÃO

O plano organiza a fase de Implementação do modelo em cascata em marcos verticais. Após cada versão do contrato, o front avança com MSW enquanto o backend implementa, o que paraleliza o trabalho (R7).

Quadro 32 – Marcos de implementação.

| Marco | Repositórios | Entregas | Critério de saída |
|---|---|---|---|
| M0 Fundação | Todos | Repositórios criados; proteções de ramo; *workflows* de CI e de release; lefthook; Release Please; fixação das versões (Java, Spring Boot, PostgreSQL, Node); esqueleto que sobe no Compose com TLS. | CI verde nos quatro repositórios; `compose up` com *health checks* saudáveis. |
| M1 Contrato 0.1 | `api-contract` | Autenticação e usuários no OpenAPI; esquema de erros e paginação; publicação dos pacotes; Prism. | Pacotes Maven e npm publicados; `oasdiff` e Spectral ativos. |
| M2 Acesso | `backend`, `front` | Migrações base; `auth` completo; usuários; administrador inicial; *layout*, *login*, sessão e tela de usuários; decisão da biblioteca de tabelas. | CA-10, CA-11, CA-12 e CA-15; login e CRUD de usuários ponta a ponta. |
| M3 Cadastros | Todos | Cooperandos e clientes (RF--COO, RF-CLI), com `ListPage` e `FormPage` reutilizáveis. | CRUDs funcionando; RN14 testada. |
| M4 Contratos e pagamentos | Todos | Contratos, máquina de estados, cronograma, baixa, repasse e estorno (RF-CTR, RF-PAG); detalhe do contrato com ações por papel. | CA-01 a CA-09; matriz de permissões testada. Marco de maior risco. |
| M5 Despesas e relatórios | Todos | Despesas, extrato, resumo, inadimplência e painel (RF-DSP, RF-REL, RF-WEB-08 e 09). | Requisitos essenciais e importantes implementados. |
| M6 Endurecimento e entrega | Todos | Teste ponta a ponta completo; k6; Lighthouse CI; backup e restauração (CA-13); documentação; *release* 1.0.0 nos quatro repositórios. | Critérios de aceitação cumpridos; RNF revisados verificados. |

Fonte: Elaborado pelos autores.

# 14 RISCOS DE DESIGN

Os riscos R1 a R7 da especificação continuam válidos. Acrescentam-se:

Quadro 33 – Riscos adicionais.

| Risco | Descrição | Mitigação |
|---|---|---|
| R8 | Deriva entre quatro repositórios (versões incompatíveis do contrato). | Contrato como fonte; oasdiff; Schemathesis no backend; compilação do front contra tipos gerados; teste ponta a ponta no repositório de infraestrutura. |
| R9 | Alteração incompatível no contrato propaga-se a dois consumidores. | Política de versionamento (Seção 4.5); `deprecated` antes de remover; PRs coordenados. |
| R10 | Automação de *release* combinada com Git Flow é incomum e pode travar o fluxo (PR de retorno, PR de release). | Fluxo descrito e automatizado (Seção 3.2); experimentar no M0 com repositórios vazios antes de haver código. |
| R11 | Leitura de pacotes do GitHub Packages exige *token* também para pacotes públicos. | `GITHUB_TOKEN` no CI; *token* pessoal com leitura de pacotes documentado no README; alternativa: publicar em registros públicos (PD02). |
| R12 | Complexidade de *cookies*, CSRF e renovação de *tokens* na SPA. | Cliente HTTP único; testes CA-10, CA-11 e CA-12; Spring Security com configuração padrão para SPAs. |
| R13 | Compatibilidade das bibliotecas com versões *major* recentes (Spring Boot 4, Jackson 3, Mantine 9, Vite). | Fixar versões no M0; Dependabot agrupado; verificar o gerador do contrato com `useSpringBoot4` e `useJackson3`. |
| R14 | Testcontainers e *builds* de imagem alongam o CI. | Cache de dependências; testes pesados só em PR; paralelismo de *jobs*. |
| R15 | Backup existente mas nunca restaurado. | `restore-check` no CI e na rotina; CA-13. |
| R16 | Certificado ACME não emitido (domínio, portas ou limites). | Volume persistente para `caddy_data`; teste prévio com o ambiente de homologação do Let's Encrypt; *runbook*. |

Fonte: Elaborado pelos autores.

# 15 PENDÊNCIAS E PREMISSAS DE DESIGN

As premissas abaixo foram adotadas para permitir o design e devem ser confirmadas na revisão deste documento, assim como foi feito com P1 a P8 na especificação.

Quadro 34 – Pendências e premissas.

| ID | Pendência ou premissa | Adotado até a confirmação |
|---|---|---|
| PD01 | Nome da organização no GitHub (escopo dos pacotes e das imagens). | Placeholder `ORG`. |
| PD02 | Aceitar *token* pessoal para ler pacotes no GitHub Packages ou publicar em registros públicos (Maven Central e npmjs). | GitHub Packages. |
| PD03 | Pacote base Java e `groupId`. | `br.ufla.cooperativa` e `br.ufla.gcc188`. |
| PD04 | Política de senha: tamanho mínimo (12 ou 15) e bloqueio (5 falhas, 15 min). | 12 caracteres; bloqueio como descrito. |
| PD05 | Troca obrigatória de senha no primeiro acesso e após redefinição pelo administrador. | Obrigatória. |
| PD06 | Biblioteca de tabelas compatível com Mantine 9. | A decidir no M2. |
| PD07 | *Hotfix* entra por `develop` e é promovido, em vez de ir direto a `master`, para respeitar "somente `develop` para `master`". | Por `develop`. |
| PD08 | Um revisor obrigatório por PR, com equipe de duas pessoas. | Um revisor que não seja o autor. |
| PD09 | Local dos testes ponta a ponta. | `gcc188-infra/e2e`. |
| PD10 | Recuperação de senha por e-mail (SMTP, Mailpit em desenvolvimento) após a 1.0. | Fora da v1. |
| PD11 | Versão *major* do PostgreSQL. | A mais recente estável, fixada no M0. |
| PD12 | Spring Boot 4.1 ou 4.2 (prevista para novembro de 2026). | A estável vigente no M0. |
| PD13 | Hospedagem e domínio da demonstração, se houver. | Somente local. |
| PD14 | *Collation* ICU para ordenação em português. | Padrão da imagem, a verificar. |
| PD15 | D38 a D42 (nomes JSON em `snake_case`, dinheiro como número, `HALF_UP`, eventos de domínio, Caddy), propostas do projetista. | Adotadas. |

Fonte: Elaborado pelos autores.

**Gestão de mudanças.** Vale o fluxo da especificação: solicitação registrada, análise de impacto (agora incluindo contrato, backend, front e infraestrutura), aprovação e atualização por PR em ramo `docs/*`.

## REFERÊNCIAS


BROWN, Simon. **The C4 model for visualising software architecture**. [S. l.]: [s. n.], [s. d.]. Disponível em: https://c4model.com/. Acesso em: 5 nov. 2026.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. [S. l.]: [s. n.], [s. d.]. Disponível em: https://www.conventionalcommits.org/. Acesso em: 5 nov. 2026.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. **Accelerate**: the science of lean software and DevOps. Portland: IT Revolution Press, 2018.

FOWLER, Martin. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GOOGLEAPIS. **Release Please**. [S. l.]: GitHub, [s. d.]. Disponível em: https://github.com/googleapis/release-please. Acesso em: 5 nov. 2026.

ISO/IEC/IEEE. **ISO/IEC/IEEE 42010:2022**: software, systems and enterprise – architecture description. Geneva: ISO, 2022.

NIST. **Digital identity guidelines**: authentication and authenticator management (SP 800-63B-4). Gaithersburg: National Institute of Standards and Technology, 2025. Disponível em: https://pages.nist.gov/800-63-4/. Acesso em: 5 nov. 2026.

NOTTINGHAM, Mark; WILDE, Erik; DALAL, Sanjay. **RFC 9457**: problem details for HTTP APIs. [S. l.]: Internet Engineering Task Force, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 5 nov. 2026.

NYGARD, Michael. **Documenting architecture decisions**. [S. l.]: Cognitect, 2011. Disponível em: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 5 nov. 2026.

OPENAPI INITIATIVE. **OpenAPI Specification v3.1.1**. [S. l.]: Linux Foundation, 2024. Disponível em: https://spec.openapis.org/oas/v3.1.1.html. Acesso em: 5 nov. 2026.

OWASP. **Cheat Sheet Series**: Cross-Site Request Forgery Prevention; JSON Web Token for Java; Password Storage. [S. l.]: OWASP Foundation, [s. d.]. Disponível em: https://cheatsheetseries.owasp.org/. Acesso em: 5 nov. 2026.

OWASP. **Application Security Verification Standard (ASVS)**. Versão 5.0. [S. l.]: OWASP Foundation, 2025. Disponível em: https://owasp.org/www-project-application-security-verification-standard/. Acesso em: 5 nov. 2026.

PRESTON-WERNER, Tom. **Semantic Versioning 2.0.0**. [S. l.]: [s. n.], 2013. Disponível em: https://semver.org/. Acesso em: 5 nov. 2026.

SPRING. **Spring Boot reference documentation**. [S. l.]: Broadcom, [s. d.]. Disponível em: https://docs.spring.io/spring-boot/. Acesso em: 5 nov. 2026.

WIGGINS, Adam. **The twelve-factor app**. [S. l.]: [s. n.], 2017. Disponível em: https://12factor.net/. Acesso em: 5 nov. 2026.